International Collegiate Programming Contest

ACM-ICPC REFERENCE MANUAL

Team Notebook

Institution:

VNU-HCM High School for the Gifted

Team:

HSG-sicula

October 4, 2026

Mục lục

1 Environment Setup . . . 2
1.1 Checklist & Code::Blocks Setup . . . 2
1.2 Source Code Hashing (FNV-1a) . . . 3
1.3 Template & Fast I/O . . . 3
1.4 Custom Fast I/O Extensions . . . 3
1.5 Optimization & Diagnostic Pragmas . . . 3
2 Data Structures . . . 3
2.1 Chtholly Tree (Old Driver Tree) . . . 3
2.2 DSU with Rollback . . . 3
2.3 Linked List (Pointers) . . . 4
2.4 Binary Grouping for Dynamic Insertion . . . 4
2.5 Fenwick Tree with lower/upper bound . . . 4
2.6 Mo's Algorithm . . . 4
2.7 Mo's Algorithm on Trees . . . 4
2.8 Mo's Algorithm with Updates (3D Mo) . . . 5
2.9 Segment Tree 2N (Iterative) . . . 5
2.10 Persistent Segment Tree . . . 5
2.11 Splay Tree . . . 5
2.12 Treap . . . 6
2.13 Implicit Treap . . . 6
2.14 Wavelet Tree . . . 6
3 Graphs and Trees . . . 7
3.1 2-SAT Solver . . . 7
3.2 Articulation Points & Bridges . . . 7
3.3 Bellman-Ford . . . 7
3.4 Centroid Decomposition . . . 7
3.5 Dijkstra Algorithm (Dense Graph) . . . 7
3.6 Finding Bridges Online . . . 8
3.7 Dinic Algorithm . . . 8
3.8 Gomory-Hu Tree . . . 8
3.9 Min Cost Max Flow (MCMF) . . . 9
3.10 Flow with Lower Bound . . . 9
3.11 Floyd-Warshall & Dynamic Update . . . 9
3.12 General Matching (Edmonds' Blossom) . . . 9
3.13 Heavy-Light Decomposition (HLD) . . . 10
3.14 Hopcroft-Karp Algorithm . . . 10
3.15 Hungarian Algorithm (Kuhn-Munkres) . . . 10
3.16 Kruskal Reconstruction Tree (KRT) . . . 11
3.17 Link-Cut Tree . . . 11
3.18 Boruvka's Algorithm . . . 11
3.19 Prim's Algorithm . . . 12
3.20 Tarjan Algorithm . . . 12
3.21 Virtual Tree (Auxiliary Tree) . . . 12
4 Math . . . 12
4.1 Chinese Remainder Theorem (CRT) . . . 12
4.2 Euler's Totient Function (Phi) . . . 12
4.3 Extended Euclidean . . . 13
4.4 FFT & NTT . . . 13
4.5 Matrix Exponentiation Template . . . 13
4.6 Miller-Rabin Primality Test & Pollard's Rho . . . 14
4.7 Stars and Bars (Chia kẹo Euler) . . . 14
4.8 Binary GCD (Stein's Algorithm) . . . 14
4.9 Stern-Brocot Tree . . . 14
4.10 Tonelli-Shanks Algorithm . . . 15
5 Strings . . . 15
5.1 Aho-Corasick Automaton . . . 15
5.2 KMP Algorithm . . . 15
5.3 Manacher's Algorithm . . . 15
5.4 Palindrome Tree (Eertree) . . . 15
5.5 String Double Hashing . . . 16
5.6 Suffix Array . . . 16
5.7 Suffix Automaton (SAM) . . . 16
5.8 Trie . . . 16
5.9 Z-Algorithm . . . 17
6 Bit Manipulation . . . 17
6.1 Binary Trie (Bit Trie) . . . 17
6.2 Bitwise Hacks . . . 17
6.3 XOR Basis . . . 17
7 DPs . . . 17
7.1 Aliens Trick (WQS Binary Search) . . . 17
7.2 Bitmask DP for Set Partitioning . . . 17
7.3 Digit DP . . . 18
7.4 Divide & Conquer Optimization . . . 18
7.5 Knuth Optimization . . . 18
7.6 Sum Over Subsets DP (SOS DP) . . . 18
8 Geometry . . . 18
8.1 Geometry Templates (Integer & Double) . . . 18
8.2 Convex Hull (Andrew's Monotone Chain) . . . 19
8.3 Incenter & Circumcenter of a Triangle . . . 19
8.4 Li Chao Tree . . . 19
8.5 LineContainer (CHT) . . . 19
8.6 Minkowski Sum . . . 19
8.7 Polygon Area (Shoelace Formula) . . . 19
8.8 Smallest Enclosing Circle (Welzl's Algorithm) . . . 19
9 Misc . . . 20
9.1 Custom Hash for hashmap & hashset . . . 20
9.2 Hill Climbing (Heuristics) . . . 20
9.3 Simulated Annealing (Heuristics) . . . 20
9.4 ModInt Template . . . 20
9.5 Policy-Based Data Structures (PBDS) . . . 21
9.6 Stress Tester . . . 21
10 Classic . . . 21
10.1 Bounded Knapsack with Binary Grouping . . . 21
10.2 CSES Counting Tilings . . . 21
10.3 Closest Pair of Points . . . 21
10.4 De Bruijn Sequence . . . 22
11 Formulas . . . 22
11.1 Essential Theorems . . . 22
11.2 Essential Formulas . . . 22
11.3 Cheatsheet . . . 23
11.4 Vocabulary . . . 24

1 Environment Setup

1.1 Checklist & Code::Blocks Setup

Contest preparation, environment validation, and shortcuts.

Contest Preparation & Environment Verification:

- Bring ID, credentials, pens, and printed Team Notebook.
- Verify system time, keyboard layout, and workspace comfort.
- Enable C++17/14:
  `Settings -> Compiler -> Compiler Settings -> Flag "-std=c++17".`
- Add compiler optimization & warning flags: `-O3`, `-Wall`, `-Wextra`, `-Wshadow`.
- Enable auto-save:
  `Settings -> Environment -> Autosave -> Check every 1 min.`
- Snippets: `Settings -> Editor -> Abbreviations`. Expands with `Ctrl + J`.

Essential Code::Blocks Shortcuts:

- `Ctrl + Shift + C`: Comment selected block.
- `Ctrl + Shift + X`: Uncomment selected block.
- `Ctrl + F9` / `Ctrl + F10` / `F9`: Compile / Run / Both.

- `Ctrl + Space` / `Ctrl + Shift + Space`: Autocomplete / Parameter tips.
- `Alt + G`: Jump to declaration or definition.
- `Alt + Up/Down`: Move selected lines up or down.
- `Ctrl + E`: Select next occurrence of keyword (Multi-select).
- `Ctrl + Click`: Multi-edit at custom positions.
- `Ctrl + D`: Duplicate selected text or current line.
- `Ctrl + T`: Swap current line with the line above it.

1.2 Source Code Hashing (FNV-1a)

Read from `code.cpp` and print an 8-character hex hash.

```python
# Python
s = "".join(open("code.cpp").read().split())
h = 14695981039346656037
for c in s: h = ((h ^ ord(c)) * 1099511628211) & 0xFFFFFFFFFFFFFFFF
print(f"Hash: {h & 0xFFFFFFFF:08X}")
```

```cpp
// C++
#include <bits/stdc++.h>
using namespace std;
int main() {
    ifstream f("code.cpp");
    string s = "", l;
    while (getline(f, l)) for (char c : l) if (!isspace(c)) s += c;
    unsigned long long h = 14695981039346656037ULL;
    for (char c : s) h = (h ^ c) * 1099511628211ULL;
    cout << hex << uppercase << setw(8) << setfill('0') << (h & 0
     xFFFFFFFF) << "\n";
}
```

Note that `code.cpp` is the file containing your source code.

1.3 Template & Fast I/O

Compile command:

```
g++ -DLOCAL -O3 -std=c++14 main.cpp -o main
```

```cpp
/* Hash: 16571137 */
#include <bits/stdc++.h>
using namespace std;

void solve() {}

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    int t = 1; cin >> t;
    while (t--) solve();
    return 0;
}
```

1.4 Custom Fast I/O Extensions

Low-level fast I/O using unlocked character reading. Handles positive integers. Returns -1 on EOF.

```cpp
/* Hash: 4C6E8D88 */
#ifdef _WIN32
#define GETCHAR _getchar_nolock
#else
#define GETCHAR getchar_unlocked
#endif

template <typename T = int>
inline bool readNum(T &x) {
    int ch = GETCHAR(); bool neg = false;
    while (ch != EOF && (ch < '0' || ch > '9') && ch != '-') ch =
     GETCHAR();
    if (ch == EOF) return false;
    if (ch == '-') { neg = true; ch = GETCHAR(); }
    x = 0;
    while (ch >= '0' && ch <= '9') { x = x * 10 + (ch - '0'); ch =
     GETCHAR(); }
    if (neg) x = -x;
    return true;
}
```

1.5 Optimization & Diagnostic Pragmas

Place at the absolute top of the source file.

```cpp
/* Hash: 1C6B12B2 */
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wpragmas"
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wshadow"
// Your code
#pragma GCC diagnostic pop
#pragma GCC diagnostic pop
```

2 Data Structures

2.1 Chtholly Tree (Old Driver Tree)

```cpp
/* Hash: 5680148A */
struct Node {
    int l, r;
    mutable long long val;

    Node(int l, int r = -1, long long val = 0) : l(l), r(r), val(
     val) {}

    bool operator<(const Node& o) const { return l < o.l; }
};


struct ChthollyTree {
    set<Node> odt;

    void init(const vector<long long>& a) {
        odt.clear();
        for (int i = 1; i < a.size(); i++) {
            odt.insert(Node(i, i, a[i]));
        }
    }

    auto split(int pos) {
        auto it = odt.lower_bound(Node(pos));
        if (it != odt.end() && it->l == pos) return it;

        --it;
        if (it->r < pos) return odt.end();

        int l = it->l, r = it->r;
        long long v = it->val;

        odt.erase(it);
        odt.insert(Node(l, pos - 1, v));
        return odt.insert(Node(pos, r, v)).first;
    }

    void assign(int l, int r, long long val) {
        auto itr = split(r + 1); // split(r + 1) first
        auto itl = split(l);

        odt.erase(itl, itr);
        odt.insert(Node(l, r, val));
    }

    void add_range(int l, int r, long long val) {
        auto itr = split(r + 1);
        auto itl = split(l);

        for (; itl != itr; ++itl) {
            itl->val += val;
        }
    }
};
```

2.2 DSU with Rollback

Complexity: $O(\log N)$. Path compression disabled. Use `get_time()` to save checkpoint and `rollback(t)` to revert.

```cpp
/* Hash: 5F4C728D */
struct DSU_Rollback {
    vector<int> parent, sz;
    vector<pair<int, int>> history;

    DSU_Rollback(int n) {
        parent.resize(n + 1); sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find_set(int v) {
        while (v != parent[v]) v = parent[v];
        return v;
    }

    int get_time() { return history.size(); }

    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a == b) return false;

        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({b, parent[b]});
        history.push_back({~a, sz[a]});

        parent[b] = a; sz[a] += sz[b];
        return true;
    }

    void rollback(int t) {
        while (history.size() > t) {
            auto [u, val] = history.back(); history.pop_back();
            if (u < 0) sz[~u] = val;
            else parent[u] = val;
        }
```

```
    }
};
```

2.3 Linked List (Pointers)

Dynamic doubly linked list supporting $O(1)$ deletion, restoration, and splicing using pointers.

```
/* Hash: DD238EF4 */
struct Node {
    int val;
    Node *L = nullptr, *R = nullptr;
    Node(int v) : val(v) {}
};
struct LinkedList {
    Node *head = nullptr, *tail = nullptr;
    void insertTail(int v) {
        Node* n = new Node(v);
        if (!head) { head = tail = n; }
        else { tail->R = n; n->L = tail; tail = n; }
    }
    void remove(Node* n) {
        if (n->L) n->L->R = n->R;
        else head = n->R;
        if (n->R) n->R->L = n->L;
        else tail = n->L;
    }
    void restore(Node* n) {
        if (n->L) n->L->R = n;
        else head = n;
        if (n->R) n->R->L = n;
        else tail = n;
    }
    void splice(LinkedList& other) {
        if (!other.head) return;
        if (!head) { head = other.head; tail = other.tail; }
        else { tail->R = other.head; other.head->L = tail; tail =
     other.tail; }
        other.head = other.tail = nullptr;
    }
};
```

2.4 Binary Grouping for Dynamic Insertion

Converts static data structures (build $O(f(N))$) into dynamic online variants supporting insertions in amortized $O(\frac{f(N)\log N}{N})$ and queries in $O(g(N)\log N)$.

```
/* Hash: 4860BFA0 */
struct StaticACAM {
    vector<string> strings;
    void insert(const string& s) { strings.push_back(s); }
    int size() const { return strings.size(); }
    void build() {}
    long long query(const string& s) { return 0; }
};

struct DynamicACAM {
    vector<StaticACAM> acams;

    void insert(const string& s) {
        StaticACAM cur; cur.insert(s);
        acams.push_back(move(cur));
        while (acams.size() >= 2 && acams.back().size() == acams[
     acams.size() - 2].size()) {
            StaticACAM rhs = move(acams.back()); acams.pop_back();
            StaticACAM lhs = move(acams.back()); acams.pop_back();
            for (const auto& str : rhs.strings) lhs.insert(str);
            lhs.build();
            acams.push_back(move(lhs));
        }
    }

    long long query(const string& s) {
        long long ans = 0;
        for (auto& ac : acams) ans += ac.query(s);
        return ans;
    }
};
```

Handles dynamic element deletions by initializing two separate `DynamicACAM` instances (one for insertions, one for deletions) and tracking the net difference.

2.5 Fenwick Tree with lower/upper bound

$O(\log N)$ binary lifting to find the first index where prefix sum $\geq$ or $>$ target. Works only for non-negative frequencies.

```
/* Hash: 5C674969 */
struct BIT {
    int n; vector<int> tree;
    BIT(int n) : n(n), tree(n + 1, 0) {}

    void update(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }

    int lower_bound(int sum) {
        int idx = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (idx + i <= n && tree[idx + i] < sum) {
                idx += i; sum -= tree[idx];
            }
        }
        return idx + 1;
    }

    int upper_bound(int sum) {
        int idx = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (idx + i <= n && tree[idx + i] <= sum) {
                idx += i; sum -= tree[idx];
            }
        }
        return idx + 1;
    }
};
```

2.6 Mo's Algorithm

Offline queries sorted by $\sqrt{N}$-sized blocks. Total time $O((N+Q)\sqrt{N})$. Set BLOCK_SIZE $\approx N/\sqrt{Q}$.

```
/* Hash: 42252FA6 */
const int BLOCK_SIZE = 450;

struct Query {
    int l, r, id;
    bool operator<(const Query& other) const {
        int b1 = l / BLOCK_SIZE, b2 = other.l / BLOCK_SIZE;
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? (r < other.r) : (r > other.r);
    }
};

long long current_answer = 0;
vector<int> cnt;

void add(int idx, const vector<int>& a) {}
void remove(int idx, const vector<int>& a) {}

vector<long long> solve_mo(vector<Query>& queries, const vector<int
     >& a) {
    sort(queries.begin(), queries.end());
    cnt.assign(1000005, 0); current_answer = 0;

    vector<long long> answers(queries.size());
    int cur_l = 0, cur_r = -1;

    for (const auto& q : queries) {
        while (cur_l > q.l) add(--cur_l, a);
        while (cur_r < q.r) add(++cur_r, a);
        while (cur_l < q.l) remove(cur_l++, a);
        while (cur_r > q.r) remove(cur_r--, a);
        answers[q.id] = current_answer;
    }
    return answers;
}
```

2.7 Mo's Algorithm on Trees

Offline queries on trees using Euler Tour (flattening the tree into an array). Set BLOCK_SIZE $\approx \frac{2N}{\sqrt{Q}}$ for an optimal time complexity of $O((N+Q)\sqrt{N})$.

```
/* Hash: 507B10DB */
const int BLOCK = 450;
const int MAXN = 100005;

struct Query {
    int l, r, lc, id;
    bool operator<(const Query& o) const {
        int b1 = l / BLOCK, b2 = o.l / BLOCK;
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? (r < o.r) : (r > o.r);
    }
};

int n, q, timer;
int st[MAXN], en[MAXN], id[2 * MAXN], depth[MAXN], up[MAXN][20];
vector<int> adj[MAXN];
bool vis[MAXN];

void dfs(int u, int p) {
    st[u] = ++timer; id[timer] = u;
    up[u][0] = p;
    for (int i = 1; i < 20; ++i) up[u][i] = up[up[u][i-1]][i-1];
    for (int v : adj[u]) {
        if (v != p) { depth[v] = depth[u] + 1; dfs(v, u); }
    }
    en[u] = ++timer; id[timer] = u;
}

int get_lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    for (int i = 19; i >= 0; --i) {
        if (depth[u] - (1 << i) >= depth[v]) u = up[u][i];
    }
    if (u == v) return u;
    for (int i = 19; i >= 0; --i) {
        if (up[u][i] != up[v][i]) { u = up[u][i]; v = up[v][i]; }
    }
    return up[u][0];
}

void check(int u) {
    if (vis[u]) { /* remove_from_data_structure(u); */ }
    else { /* add_to_data_structure(u); */ }
```

```
    vis[u] ^= 1;
}

vector<long long> solve_mo_tree(vector<Query>& queries) {
    timer = 0; dfs(1, 1);
    for (auto& q : queries) {
        int u = q.l, v = q.r; // Here q.l and q.r store original
     tree node indices
        int lca = get_lca(u, v);
        if (st[u] > st[v]) swap(u, v);
        if (lca == u) {
            q.l = st[u]; q.r = st[v]; q.lc = 0;
        } else {
            q.l = en[u]; q.r = st[v]; q.lc = lca;
        }
    }
    sort(queries.begin(), queries.end());

    vector<long long> answers(queries.size());
    int cur_l = 1, cur_r = 0;
    for (const auto& q : queries) {
        while (cur_l > q.l) check(id[--cur_l]);
        while (cur_r < q.r) check(id[++cur_r]);
        while (cur_l < q.l) check(id[cur_l++]);
        while (cur_r > q.r) check(id[cur_r--]);
        if (q.lc) check(q.lc);
        answers[q.id] = 0; // fetch current answer here
        if (q.lc) check(q.lc);
    }
    return answers;
}
```

- Convert path queries into range queries on the `id` array size $2N$ (1-indexed).
- If LCA is one of the endpoints, the range is `[st[u], st[v]]`. Otherwise, the range is `[en[u], st[v]]` plus the isolated `lca` node.
- The function `check(u)` toggles the presence of node u, handling insertions and deletions symmetrically.

2.8 Mo's Algorithm with Updates (3D Mo)

Offline queries with point updates sorted by block-based 3D coordinates. Set BLOCK_SIZE $\approx N^{2/3}$ for an optimal time complexity of $O(N^{5/3})$.

```
/* Hash: EE0E96A8 */
const int BLOCK = 2150;

struct Query {
    int l, r, t, id;
    bool operator<(const Query& o) const {
        int bl1 = l / BLOCK, bl2 = o.l / BLOCK;
        if (bl1 != bl2) return bl1 < bl2;
        int br1 = r / BLOCK, br2 = o.r / BLOCK;
        if (br1 != br2) return br1 < br2;
        return t < o.t;
    }
};

struct Update {
    int pos, old_val, new_val;
};

long long current_answer = 0;
vector<int> cnt;

void add(int idx, const vector<int>& a) {}
void remove(int idx, const vector<int>& a) {}

void apply_update(int t, int cur_l, int cur_r, vector<int>& a,
     const vector<Update>& updates) {
    int pos = updates[t].pos;
    if (pos >= cur_l && pos <= cur_r) {
        remove(pos, a);
        a[pos] = updates[t].new_val;
        add(pos, a);
    } else {
        a[pos] = updates[t].new_val;
    }
}

void rollback_update(int t, int cur_l, int cur_r, vector<int>& a,
     const vector<Update>& updates) {
    int pos = updates[t].pos;
    if (pos >= cur_l && pos <= cur_r) {
        remove(pos, a);
        a[pos] = updates[t].old_val;
        add(pos, a);
    } else {
        a[pos] = updates[t].old_val;
    }
}

vector<long long> solve_mo_update(vector<Query>& queries, vector<
     Update>& updates, vector<int>& a) {
    sort(queries.begin(), queries.end());
    cnt.assign(1000005, 0); current_answer = 0;
    vector<long long> answers(queries.size());

    int cur_l = 0, cur_r = -1, cur_t = 0;
    for (const auto& q : queries) {
        while (cur_t < q.t) apply_update(cur_t++, cur_l, cur_r, a,
     updates);
        while (cur_t > q.t) rollback_update(--cur_t, cur_l, cur_r,
     a, updates);
        while (cur_l > q.l) add(--cur_l, a);
        while (cur_r < q.r) add(++cur_r, a);
        while (cur_l < q.l) remove(cur_l++, a);
        while (cur_r > q.r) remove(cur_r--, a);
        answers[q.id] = current_answer;
    }
    return answers;
}
```

- Separate timeline modifications into the `updates` vector (0-indexed). Assign the total number of updates prior to a query to `Query.t`.
- Tune `BLOCK` dynamically based on N and Q to fit within the time limit.

2.9 Segment Tree 2N (Iterative)

0-indexed, $O(\log N)$ point update and range query $[l, r]$. Order matters for non-commutative operations. For 1-indexed, use size `2 * (n + 1)`.

```
/* Hash: 78E8D3A6 */
struct SegTree {
    int n; vector<int> tree;
    SegTree(int n) : n(n), tree(2 * n, 0) {}

    void update(int i, int value) {
        for (tree[i += n] = value; i > 1; i >>= 1) {
            tree[i >> 1] = tree[i] + tree[i ^ 1];
        }
    }

    int query(int l, int r) {
        int res = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += tree[l++];
            if (r & 1) res += tree[--r];
        }
        return res;
    }
};
```

2.10 Persistent Segment Tree

$O(\log N)$ time and space per update. `upgrade(v, pos, val)` creates a new version from version `v`. Query version `v` via `query(roots[v], 1, n, ql, qr)`.

```
/* Hash: 80E0FB6E */
struct Node {
    int val; Node *l, *r;
    Node(int val = 0) : val(val), l(nullptr), r(nullptr) {}
    Node(Node* o) : val(o->val), l(o->l), r(o->r) {}
};

struct PersistentSegTree {
    int n; vector<Node*> roots;
    PersistentSegTree(int n) : n(n) { roots.push_back(build(1, n));
      }

    Node* build(int l, int r) {
        Node* curr = new Node(0);
        if (l == r) return curr;
        int mid = (l + r) >> 1;
        curr->l = build(l, mid); curr->r = build(mid + 1, r);
        return curr;
    }

    Node* update(Node* prev_node, int l, int r, int pos, int val) {
        Node* curr = new Node(prev_node);
        if (l == r) { curr->val += val; return curr; }
        int mid = (l + r) >> 1;
        if (pos <= mid) curr->l = update(prev_node->l, l, mid, pos,
     val);
        else curr->r = update(prev_node->r, mid + 1, r, pos, val);
        curr->val = curr->l->val + curr->r->val;
        return curr;
    }

    int query(Node* node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return node->val;
        int mid = (l + r) >> 1, res = 0;
        if (ql <= mid) res += query(node->l, l, mid, ql, qr);
        if (qr > mid) res += query(node->r, mid + 1, r, ql, qr);
        return res;
    }

    void upgrade(int version, int pos, int val) {
        roots.push_back(update(roots[version], 1, n, pos, val));
    }
};
```

2.11 Splay Tree

A self-balancing binary search tree that moves accessed nodes to the root.

```
/* Hash: 9DB7C656 */
struct SplayTree {
    struct Node {
        int ch[2] = {0, 0}, p = 0, val = 0;
    } t[100005];
    int root = 0, tot = 0;
    void rotate(int x) {
        int y = t[x].p, z = t[y].p;
        int k = (t[y].ch[1] == x);
        t[z].ch[t[z].ch[1] == y] = x; t[x].p = z;
        t[y].ch[k] = t[x].ch[k ^ 1]; t[t[x].ch[k ^ 1]].p = y;
        t[x].ch[k ^ 1] = y; t[y].p = x;
    }
    void splay(int x, int goal = 0) {
        while (t[x].p != goal) {
            int y = t[x].p, z = t[y].p;
            if (z != goal) {
                if ((t[y].ch[1] == x) ^ (t[z].ch[1] == y)) rotate(x
     );
                else rotate(y);
            }
            rotate(x);
        }
        if (!goal) root = x;
    }
    void insert(int v) {
        int u = root, p = 0;
        while (u && t[u].val != v) {
            p = u;
            u = t[u].ch[v > t[u].val];
        }
        if (u) {
            splay(u);
        } else {
            u = ++tot;
            if (p) t[p].ch[v > t[p].val] = u;
            t[u].p = p; t[u].val = v;
            splay(u);
        }
    }
};
```

2.12 Treap

Balanced BST combining BST and Heap properties. Operations run in $O(\log N)$ expected time.

```
/* Hash: DE3082B3 */
struct Node {
    int key, prior, sz;
    Node *l, *r;
    Node(int key) : key(key), prior(rng()), sz(1), l(nullptr), r(
     nullptr) {}
};

int get_sz(Node* t) { return t ? t->sz : 0; }
void upd_sz(Node* t) { if (t) t->sz = 1 + get_sz(t->l) + get_sz(t->
     r); }

void split(Node* t, int key, Node*& l, Node*& r) {
    if (!t) l = r = nullptr;
    else if (t->key <= key) {
        split(t->r, key, t->r, r);
        l = t; upd_sz(l);
    } else {
        split(t->l, key, l, t->l);
        r = t; upd_sz(r);
    }
}

void merge(Node*& t, Node* l, Node* r) {
    if (!l || !r) t = l ? l : r;
    else if (l->prior > r->prior) {
        merge(l->r, l->r, r);
        t = l; upd_sz(t);
    } else {
        merge(r->l, l, r->l);
        t = r; upd_sz(t);
    }
}

void insert(Node*& t, Node* it) {
    if (!t) t = it;
    else if (it->prior > t->prior) {
        split(t, it->key, it->l, it->r);
        t = it; upd_sz(t);
    } else {
        insert(t->key <= it->key ? t->r : t->l, it);
        upd_sz(t);
    }
}

void erase(Node*& t, int key) {
    if (!t) return;
    if (t->key == key) {
        Node* th = t; merge(t, t->l, t->r); delete th;
    } else {
        erase(key < t->key ? t->l : t->r, key); upd_sz(t);
    }
}
```

2.13 Implicit Treap

Indexed by array positions instead of keys. Supports range operations (e.g., reverse) and split/merge in $O(\log N)$ expected time.

```
/* Hash: 86AD2D63 */
struct Node {
    int val, prior, sz;
    bool rev;
    Node *l, *r;
    Node(int val) : val(val), prior(rng()), sz(1), rev(false), l(
     nullptr), r(nullptr) {}
};

int get_sz(Node* t) { return t ? t->sz : 0; }
void upd_sz(Node* t) { if (t) t->sz = 1 + get_sz(t->l) + get_sz(t->
     r); }

void push(Node* t) {
    if (t && t->rev) {
        t->rev = false;
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= true;
        if (t->r) t->r->rev ^= true;
    }
}

void split(Node* t, int k, Node*& l, Node*& r) {
    if (!t) { l = r = nullptr; return; }
    push(t);
    if (get_sz(t->l) < k) {
        split(t->r, k - get_sz(t->l) - 1, t->r, r);
        l = t; upd_sz(l);
    } else {
        split(t->l, k, l, t->l);
        r = t; upd_sz(r);
    }
}

void merge(Node*& t, Node* l, Node* r) {
    push(l); push(r);
    if (!l || !r) t = l ? l : r;
    else if (l->prior > r->prior) {
        merge(l->r, l->r, r);
        t = l; upd_sz(t);
    } else {
        merge(r->l, l, r->l);
        t = r; upd_sz(t);
    }
}

void reverse_range(Node*& t, int ql, int qr) {
    Node *l1, *r1, *l2, *r2;
    split(t, ql, l1, r1);
    split(r1, qr - ql + 1, l2, r2);
    if (l2) l2->rev ^= true;
    merge(r1, l2, r2);
    merge(t, l1, r1);
}
```

2.14 Wavelet Tree

Generalized range queries (k-th element, count $\leq k$) in $O(\log(\max A_i))$. Queries expect 1-indexed boundaries.

```
/* Hash: 60BD8FAF */
struct WaveletTree{
    int lo, hi;
    vector<int> pref;
    vector<long long> psum;
    WaveletTree *l, *r;

    WaveletTree(int *from, int *to, int L, int R){
        lo = L; hi = R;
        l = r = nullptr;
        if(from >= to) return;

        int mid = (lo + hi) >> 1;
        auto f = [&mid](int x){ return x <= mid; };

        pref.push_back(0);
        psum.push_back(0);
        for(auto it = from; it != to; ++it){
            pref.push_back(pref.back() + f(*it));
            psum.push_back(psum.back() + *it);
        }

        if(lo == hi) return;

        auto pivot = stable_partition(from, to, f);
        l = new WaveletTree(from, pivot, lo, mid);
        r = new WaveletTree(pivot, to, mid + 1, hi);
    }

    int kth(int lq, int rq, int k){
        if(lq > rq) return INF + 1;
        if(lo == hi) return lo;

        int inLeft = pref[rq] - pref[lq - 1];
        if(k <= inLeft) return l->kth(pref[lq - 1] + 1, pref[rq], k
     );
        else return r->kth(lq - pref[lq - 1], rq - pref[rq], k -
     inLeft);
    }

    int cnt_leq(int lq, int rq, int x){
        if(lq > rq) return 0;
        if(lo > x) return 0;
        if(hi <= x) return rq - lq + 1;

        return l->cnt_leq(pref[lq - 1] + 1, pref[rq], x) +
               r->cnt_leq(lq - pref[lq - 1], rq - pref[rq], x);
    }
```

```
    long long sum_leq(int lq, int rq, int x){
        if(lq > rq) return 0;
        if(lo > x) return 0;
        if(hi <= x) return psum[rq] - psum[lq - 1];

        return l->sum_leq(pref[lq - 1] + 1, pref[rq], x) +
               r->sum_leq(lq - pref[lq - 1], rq - pref[rq], x);
    }

    int cnt_eq(int lq, int rq, int x){
        if(lq > rq || hi < x || x < lo) return 0;
        if(lo == hi) return rq - lq + 1;

        int mid = (lo + hi) >> 1;
        if(x <= mid) return l->cnt_eq(pref[lq - 1] + 1, pref[rq], x
     );
        else return r->cnt_eq(lq - pref[lq - 1], rq - pref[rq], x);
    }
};
```

3 Graphs and Trees

3.1 2-SAT Solver

Solves 2-SAT in $O(V + E)$ using Tarjan's SCC. Variables are 1-indexed. Use negative sign for negation (e.g., -u).

```
/* Hash: DFA82919 */
struct TwoSat {
    int n, timer, scc_count;
    vector<vector<int>> adj;
    vector<int> tin, low, scc_id;
    vector<bool> in_stack, answer;
    stack<int> st;

    TwoSat(int n) : n(n), adj(2 * n + 1), tin(2 * n + 1, -1), low(2
      * n + 1, -1),
                    scc_id(2 * n + 1, -1), in_stack(2 * n + 1,
     false), answer(n + 1, false),
                    timer(0), scc_count(0) {}

    void add_clause(int u, int v) {
        int not_u = (u > 0) ? u + n : -u;
        int act_u = (u > 0) ? u : -u + n;
        int not_v = (v > 0) ? v + n : -v;
        int act_v = (v > 0) ? v : -v + n;
        adj[not_u].push_back(act_v); adj[not_v].push_back(act_u);
    }

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u); in_stack[u] = true;
        for (int v : adj[u]) {
            if (tin[v] == -1) {
                dfs(v); low[u] = min(low[u], low[v]);
            } else if (in_stack[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }
        if (low[u] == tin[u]) {
            scc_count++;
            while (true) {
                int v = st.top(); st.pop(); in_stack[v] = false;
                scc_id[v] = scc_count;
                if (u == v) break;
            }
        }
    }

    bool solve() {
        for (int i = 1; i <= 2 * n; i++) if (tin[i] == -1) dfs(i);
        for (int i = 1; i <= n; i++) {
            if (scc_id[i] == scc_id[i + n]) return false;
            answer[i] = scc_id[i] < scc_id[i + n];
        }
        return true;
    }
};
```

3.2 Articulation Points & Bridges

Finds cut vertices and bridges in $O(V + E)$ using DFS. Handles multiple edges correctly.

```
/* Hash: 523ED69D */
struct CutVerticesBridges {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, low;
    vector<bool> is_cut;
    vector<pair<int, int>> bridges;

    CutVerticesBridges(int n, const vector<vector<int>>& adj) :
        n(n), adj(adj), timer(0), tin(n + 1, -1), low(n + 1, -1),
     is_cut(n + 1, false) {}

    void dfs(int u, int p = -1) {
        tin[u] = low[u] = ++timer;
        int children = 0, p_count = 0;

        for (int v : adj[u]) {
            if (v == p && ++p_count == 1) continue; // Handles
     multiple edges
            if (tin[v] != -1) {
                low[u] = min(low[u], tin[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u] && p != -1) is_cut[u] = true;
                if (low[v] > tin[u]) bridges.push_back({min(u, v),
     max(u, v)});
                children++;
            }
        }
        if (p == -1 && children > 1) is_cut[u] = true;
    }

    void run() {
        for (int i = 1; i <= n; i++) if (tin[i] == -1) dfs(i);
    }
};
```

3.3 Bellman-Ford

Finds shortest paths from a single source or detects negative cycles. Time complexity: $O(V \cdot E)$.

```
/* Hash: E9D8A914 */
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;
struct Edge { int u, v; long long w; };

int n, m;
vector<Edge> edges;
vector<long long> dist;

bool bellman_ford(int src) {
    dist.assign(n + 1, INF);
    dist[src] = 0;
    for (int i = 1; i < n; i++) {
        bool any = false;
        for (auto &e : edges) {
            if (dist[e.u] == INF) continue;
            long long &ans = dist[e.v];
            if (dist[e.u] + e.w < ans) {
                ans = dist[e.u] + e.w;
                any = true;
            }
        }
        if (!any) break;
    }

    for (auto &e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) return
      true;
    }
    return false;
}
```

3.4 Centroid Decomposition

Builds a centroid tree in $O(N \log N)$ time with depth $O(\log N)$. Tree layout is stored in `parent` for upward updates.

```
/* Hash: 0DD45D00 */
struct CentroidDecomposition {
    int n; vector<vector<int>> adj;
    vector<int> sz, parent; vector<bool> removed;

    CentroidDecomposition(int n, const vector<vector<int>>&
     tree_adj) :
        n(n), adj(tree_adj), sz(n + 1), parent(n + 1, 0), removed(n
      + 1, false) {}

    int get_sz(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) if (v != p && !removed[v]) sz[u] +=
     get_sz(v, u);
        return sz[u];
    }

    int find_centroid(int u, int p, int comp_sz) {
        for (int v : adj[u]) {
            if (v != p && !removed[v] && sz[v] > comp_sz / 2)
     return find_centroid(v, u, comp_sz);
        }
        return u;
    }

    int build_tree(int u) {
        int comp_sz = get_sz(u, 0), centroid = find_centroid(u, 0,
     comp_sz);
        removed[centroid] = true;
        for (int v : adj[centroid]) {
            if (!removed[v]) parent[build_tree(v)] = centroid;
        }
        return centroid;
    }

    void init(int start_node = 1) { build_tree(start_node); }
};
```

3.5 Dijkstra Algorithm (Dense Graph)

Shortest path algorithm for dense graphs ($E \approx V^2$) running in $O(V^2)$ without using a priority queue.

```
/* Hash: A9B3031D */
const long long INF = 1e18;
```

```
vector<long long> dijkstra_dense(int source, int n, const vector<
     vector<long long>>& adj) {
    vector<long long> dist(n + 1, INF);
    vector<bool> visited(n + 1, false);

    dist[source] = 0;
    for (int i = 1; i <= n; i++) {
        int u = -1;
        for (int v = 1; v <= n; v++) {
            if (!visited[v] && (u == -1 || dist[v] < dist[u])) u =
     v;
        }

        if (dist[u] == INF) break;
        visited[u] = true;

        for (int v = 1; v <= n; v++) {
            if (adj[u][v] != INF && dist[u] + adj[u][v] < dist[v])
     {
                dist[v] = dist[u] + adj[u][v];
            }
        }
    }
    return dist;
}
```

3.6 Finding Bridges Online

Maintains 2-edge-connected components dynamically under edge insertions in $O(N \log N + M \log N)$ total time using two DSU structures.

```
/* Hash: FAA55D26 */
struct DynamicBridge {
    int bridges, lca_iter;
    vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size, last_visit;

    DynamicBridge(int n) : bridges(0), lca_iter(0), par(n),
     dsu_2ecc(n), dsu_cc(n), dsu_cc_size(n, 1), last_visit(n, 0) {
        iota(dsu_2ecc.begin(), dsu_2ecc.end(), 0);
        iota(dsu_cc.begin(), dsu_cc.end(), 0);
        fill(par.begin(), par.end(), -1);
    }

    int find_2ecc(int v) {
        if (v == -1) return -1;
        return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(
     dsu_2ecc[v]);
    }

    int find_cc(int v) {
        v = find_2ecc(v);
        return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
    }

    void make_root(int v) {
        int root = v, child = -1;
        while (v != -1) {
            int p = find_2ecc(par[v]);
            par[v] = child; dsu_cc[v] = root;
            child = v; v = p;
        }
        dsu_cc_size[root] = dsu_cc_size[child];
    }

    void merge_path(int a, int b) {
        ++lca_iter;
        vector<int> path_a, path_b;
        int lca = -1;
        while (lca == -1) {
            if (a != -1) {
                a = find_2ecc(a); path_a.push_back(a);
                if (last_visit[a] == lca_iter) { lca = a; break; }
                last_visit[a] = lca_iter; a = par[a];
            }
            if (b != -1) {
                b = find_2ecc(b); path_b.push_back(b);
                if (last_visit[b] == lca_iter) { lca = b; break; }
                last_visit[b] = lca_iter; b = par[b];
            }
        }
        for (int v : path_a) { dsu_2ecc[v] = lca; if (v == lca)
     break; --bridges; }
        for (int v : path_b) { dsu_2ecc[v] = lca; if (v == lca)
     break; --bridges; }
    }

    void add_edge(int a, int b) {
        a = find_2ecc(a); b = find_2ecc(b);
        if (a == b) return;
        int ca = find_cc(a), cb = find_cc(b);
        if (ca != cb) {
            ++bridges;
            if (dsu_cc_size[ca] > dsu_cc_size[cb]) { swap(a, b);
     swap(ca, cb); }
            make_root(a);
            par[a] = dsu_cc[a] = b;
            dsu_cc_size[cb] += dsu_cc_size[a];
        } else {
            merge_path(a, b);
        }
    }
};
```

- Initialize via `DynamicBridge db(n)` (0-indexed).
- Call `db.add_edge(u, v)` online to insert an undirected edge and incrementally update the forest structure.
- Access `db.bridges` at any execution state to fetch the current total bridge count across all components.

3.7 Dinic Algorithm

Max flow and min-cut algorithm running in $O(V^2E)$ (worst case) and $O(E\sqrt{V})$ on unit networks.

```
/* Hash: E7D782E2 */
const long long INF = 1e18;

struct Dinic {
    struct Edge { int to; long long flow, cap; int rev; };
    int n, source, sink;
    vector<int> level, ptr;
    vector<vector<Edge>> adj;

    Dinic(int n, int source, int sink) : n(n), source(source), sink
     (sink), level(n + 1), ptr(n + 1), adj(n + 1) {}

    void add_edge(int from, int to, long long cap) {
        adj[from].push_back({to, 0, cap, (int)adj[to].size()});
        adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
    }

    bool bfs() {
        fill(level.begin(), level.end(), -1);
        level[source] = 0; queue<int> q; q.push(source);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto& edge : adj[u]) {
                if (edge.cap - edge.flow > 0 && level[edge.to] ==
     -1) {
                    level[edge.to] = level[u] + 1; q.push(edge.to);
                }
            }
        }
        return level[sink] != -1;
    }

    long long dfs(int u, long long pushed) {
        if (pushed == 0 || u == sink) return pushed;
        for (int& cid = ptr[u]; cid < adj[u].size(); ++cid) {
            auto& edge = adj[u][cid]; int tr = edge.to;
            if (level[u] + 1 != level[tr] || edge.cap - edge.flow
     == 0) continue;
            long long tr_pushed = dfs(tr, min(pushed, edge.cap -
     edge.flow));
            if (tr_pushed == 0) continue;
            edge.flow += tr_pushed; adj[tr][edge.rev].flow -=
     tr_pushed;
            return tr_pushed;
        }
        return 0;
    }

    long long max_flow() {
        long long flow = 0;
        while (bfs()) {
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(source, INF)) flow +=
     pushed;
        }
        return flow;
    }

    vector<pair<int, int>> min_cut() {
        vector<pair<int, int>> cuts;
        for (int u = 0; u <= n; ++u) {
            if (level[u] == -1) continue;
            for (auto& edge : adj[u]) {
                if (edge.cap > 0 && level[edge.to] == -1) {
                    cuts.push_back({u, edge.to});
                }
            }
        }
        return cuts;
    }
};
```

- Call `max_flow()` first to compute the maximum flow from `source` to `sink`.
- The array `level` retains the reachability from `source` in the residual graph. A node v is on the source side of the cut if `level[v] != -1`.
- Call `min_cut()` after `max_flow()` to get all edges belonging to the minimum cut partition.

3.8 Gomory-Hu Tree

A tree structure representing the minimum cuts (and maximum flows) between all pairs of vertices in an undirected graph using $N - 1$ max-flow calls.

```
/* Hash: C4BF8E4C */
struct GomoryHu {
```

```
    struct Edge { int u, v; long long w; };
    int n;
    vector<Edge> tree;

    GomoryHu(int _n) : n(_n) {}

    // Requires a Dinic or PushRelabel solver instance "D"
      initialized with N nodes
    // and the original graph edges added with their respective
      capacities.
    void build(const vector<pair<pair<int, int>, long long>>& edges
     ) {
        vector<int> p(n, 0);
        for (int i = 1; i < n; ++i) {
            Dinic D(n, i, p[i]); // Replace with your standard
     Dinic(N, S, T)
            for (auto& e : edges) {
                D.add_edge(e.first.first, e.first.second, e.second)
     ;
                D.add_edge(e.first.second, e.first.first, e.second)
     ; // Undirected
            }
            long long flow = D.max_flow();
            tree.push_back({i, p[i], flow});
            for (int j = i + 1; j < n; ++j) {
                if (p[j] == p[i] && D.level[j] != -1) { // j is on
     i's side of the min-cut
                    p[j] = i;
                }
            }
        }
    }
};
```

- Initialize via `GomoryHu gh(n)` (0-indexed). Pass all graph edges to `gh.build(edges)`.
- Assumes your standard `Dinic` structure can report the min-cut partition via reachable nodes (`D.level[j] != -1` after `max_flow`).
- The output `tree` contains exactly $N-1$ edges. The all-pairs max-flow/min-cut between any u and v equals the minimum edge weight along the path connecting them in this resulting tree.

3.9 Min Cost Max Flow (MCMF)

Finds max flow with min cost in $O(F \cdot E \log V)$ using SPFA (F is the max flow). Negative costs are allowed, but no negative cycles.

```
/* Hash: D6D8EC35 */
const long long INF = 1e18;

struct MCMF {
    struct Edge { int to; long long cap, flow, cost; int rev; };
    int n, source, sink;
    vector<vector<Edge>> adj;
    vector<long long> dist;
    vector<int> parent, parent_edge;
    vector<bool> in_queue;

    MCMF(int n, int source, int sink) : n(n), source(source), sink(
     sink), adj(n + 1), dist(n + 1), parent(n + 1), parent_edge(n +
      1), in_queue(n + 1) {}

    void add_edge(int from, int to, long long cap, long long cost)
     {
        adj[from].push_back({to, cap, 0, cost, (int)adj[to].size()
     });
        adj[to].push_back({from, 0, 0, -cost, (int)adj[from].size()
      - 1});
    }

    bool spfa() {
        fill(dist.begin(), dist.end(), INF); fill(in_queue.begin(),
      in_queue.end(), false);
        queue<int> q; dist[source] = 0; q.push(source); in_queue[
     source] = true;

        while (!q.empty()) {
            int u = q.front(); q.pop(); in_queue[u] = false;
            for (int i = 0; i < adj[u].size(); i++) {
                auto& edge = adj[u][i];
                if (edge.cap - edge.flow > 0 && dist[u] + edge.cost
      < dist[edge.to]) {
                    dist[edge.to] = dist[u] + edge.cost; parent[
     edge.to] = u; parent_edge[edge.to] = i;
                    if (!in_queue[edge.to]) { q.push(edge.to);
     in_queue[edge.to] = true; }
                }
            }
        }
        return dist[sink] != INF;
    }

    pair<long long, long long> solve() {
        long long max_flow = 0, min_cost = 0;
        while (spfa()) {
            long long pushed = INF;
            for (int v = sink; v != source; v = parent[v]) {
                pushed = min(pushed, adj[parent[v]][parent_edge[v
     ]].cap - adj[parent[v]][parent_edge[v]].flow);
            }
            for (int v = sink; v != source; v = parent[v]) {
                auto& edge = adj[parent[v]][parent_edge[v]];
                edge.flow += pushed; adj[v][edge.rev].flow -=
     pushed;
                min_cost += pushed * edge.cost;
            }
            max_flow += pushed;
        }
        return {max_flow, min_cost};
    }
};
```

3.10 Flow with Lower Bound

Graph transformation for network flows with lower bounds $[L, C]$.

1. **Transform Graph:** New capacity is $C - L$. Update demands: delta$[u] -= L$, delta$[v] += L$.
2. **Super Source (S') & Sink (T'):** For each vertex u:
   - delta$[u] > 0$: Add edge (S', u) with capacity delta$[u]$.
   - delta$[u] < 0$: Add edge (u, T') with capacity $-$delta$[u]$.

Feasible Flow (Circulatory): Run Max Flow $S' \to T'$. Feasible iff max_flow == $\sum_{\text{delta}>0}$ delta. Real flow = L + current_flow.

Max/Min Flow with Source (S) and Sink (T):

1. Add edge (T, S) with capacity ∞. Run Max Flow $S' \to T'$ to check feasibility.
2. Save `init_flow` = flow on (T, S), then remove edge (T, S).
3. **Max Flow:** Run Max Flow $S \to T$. `Total` = `init_flow` + new_flow(S, T).
4. **Min Flow:** Run Max Flow $T \to S$. `Total` = `init_flow` - new_flow(T, S).

3.11 Floyd-Warshall & Dynamic Update

All-pairs shortest paths in $O(N^3)$ and dynamic vertex insertion in $O(N^2)$.

```
/* Hash: C30EF4B1 */
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;
int n;
long long dist[505][505];

void floyd_warshall() {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[
     k][j]);
}

// Fast update in O(N^2) when adding a NEW vertex 'u'
void add_vertex(int u) {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (dist[i][u] < INF && dist[u][j] < INF)
                dist[i][j] = min(dist[i][j], dist[i][u] + dist[u][j
     ]);
}
```

Initialize `dist[i][i] = 0`. For offline queries where vertices are deleted, reverse the query order and use `add_vertex` instead. Negative cycle exists if any `dist[i][i] < 0`.

3.12 General Matching (Edmonds' Blossom)

Finds a maximum matching in general graphs, running in $\mathcal{O}(V^3)$ time complexity.

```
/* Hash: F37D049B */
struct Blossom {
    int n, timer;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<bool> inq, inb, blo;

    Blossom(int n) : n(n), g(n + 1), match(n + 1, 0), p(n + 1),
     base(n + 1), q(n + 1) {}

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

```
int lca(int a, int b) {
    static vector<int> used(n + 1, 0);
    timer++;
    for (;; swap(a, b)) {
        if (a) {
            a = base[a];
            if (used[a] == timer) return a;
            used[a] = timer;
            a = p[match[a]];
        }
    }
}

void mark(int v, int b, int children) {
    for (; base[v] != b; v = p[match[v]]) {
        blo[base[v]] = blo[base[match[v]]] = true;
        p[v] = children;
        children = match[v];
    }
}

int bfs(int root) {
    fill(p.begin(), p.end(), 0);
    inq.assign(n + 1, false);
    for (int i = 1; i <= n; i++) base[i] = i;
    int ql = 0, qr = 0;
    q[qr++] = root;
    inq[root] = true;

    while (ql < qr) {
        int u = q[ql++];
        for (int v : g[u]) {
            if (base[u] == base[v] || match[u] == v) continue;
            if (v == root || (match[v] && p[match[v]])) {
                int b = lca(u, v);
                blo.assign(n + 1, false);
                mark(u, b, v);
                mark(v, b, u);
                for (int i = 1; i <= n; i++) {
                    if (blo[base[i]]) {
                        base[i] = b;
                        if (!inq[i]) {
                            inq[i] = true;
                            q[qr++] = i;
                        }
                    }
                }
            } else if (!p[v]) {
                p[v] = u;
                if (!match[v]) return v;
                v = match[v];
                inq[v] = true;
                q[qr++] = v;
            }
        }
    }
    return 0;
}

int solve() {
    int max_match = 0;
    for (int i = 1; i <= n; i++) {
        if (!match[i]) {
            int v = bfs(i);
            while (v) {
                int pv = p[v], nv = match[pv];
                match[v] = pv;
                match[pv] = v;
                v = nv;
            }
        }
    }
    for (int i = 1; i <= n; i++) if (match[i]) max_match++;
    return max_match / 2;
}
```

3.13 Heavy-Light Decomposition (HLD)

Path queries/updates in $O(\log^2 N)$ with Segment Tree. Continuous range per heavy path mapped via `pos`. For edge queries, use `pos[u] + 1` instead of `pos[u]` at the final step.

```
/* Hash: EA80E6BB */
struct HLD {
    int n, timer; vector<vector<int>> adj;
    vector<int> parent, depth, heavy, head, pos;

    HLD(int n, const vector<vector<int>>& tree_adj) :
        n(n), timer(0), adj(tree_adj), parent(n + 1), depth(n + 1),
      heavy(n + 1, -1), head(n + 1), pos(n + 1) {}

    int dfs_sz(int u, int p, int d) {
        int size = 1, max_c_size = 0; parent[u] = p; depth[u] = d;
        for (int v : adj[u]) if (v != p) {
            int c_size = dfs_sz(v, u, d + 1); size += c_size;
            if (c_size > max_c_size) { max_c_size = c_size; heavy[u
     ] = v; }
        }
        return size;
    }

    void decompose(int u, int h) {
        head[u] = h; pos[u] = ++timer;
        if (heavy[u] != -1) decompose(heavy[u], h);
        for (int v : adj[u]) if (v != parent[u] && v != heavy[u])
      decompose(v, v);
    }

    void init(int root = 1) { dfs_sz(root, 0, 0); decompose(root,
      root); }

    template<typename SegTree>
    int query_path(int u, int v, SegTree& seg) {
        int res = 0;
        for (; head[u] != head[v]; v = parent[head[v]]) {
            if (depth[head[u]] > depth[head[v]]) swap(u, v);
            res += seg.query(pos[head[v]], pos[v]);
        }
        if (depth[u] > depth[v]) swap(u, v);
        return res + seg.query(pos[u], pos[v]);
    }

    template<typename SegTree>
    void update_path(int u, int v, int val, SegTree& seg) {
        for (; head[u] != head[v]; v = parent[head[v]]) {
            if (depth[head[u]] > depth[head[v]]) swap(u, v);
            seg.update(pos[head[v]], pos[v], val);
        }
        if (depth[u] > depth[v]) swap(u, v);
        seg.update(pos[u], pos[v], val);
    }
};
```

3.14 Hopcroft-Karp Algorithm

Maximum cardinality matching in bipartite graph in $O(E\sqrt{V})$. Vertices U and V are 1-indexed.

```
/* Hash: FB7F7A1B */
const int INF = 1e9;

struct HopcroftKarp {
    int n, m; vector<vector<int>> adj;
    vector<int> pair_u, pair_v, dist;

    HopcroftKarp(int n, int m) : n(n), m(m), adj(n + 1), pair_u(n +
      1, 0), pair_v(m + 1, 0), dist(n + 1, 0) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (pair_u[u] == 0) { dist[u] = 0; q.push(u); }
            else dist[u] = INF;
        }
        dist[0] = INF;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] < dist[0]) {
                for (int v : adj[u]) if (dist[pair_v[v]] == INF) {
                    dist[pair_v[v]] = dist[u] + 1; q.push(pair_v[v
     ]);
                }
            }
        }
        return dist[0] != INF;
    }

    bool dfs(int u) {
        if (!u) return true;
        for (int v : adj[u]) {
            if (dist[pair_v[v]] == dist[u] + 1 && dfs(pair_v[v])) {
                pair_v[v] = u; pair_u[u] = v; return true;
            }
        }
        dist[u] = INF; return false;
    }

    int max_matching() {
        int matching = 0;
        while (bfs()) {
            for (int u = 1; u <= n; u++) if (pair_u[u] == 0 && dfs(
      u)) matching++;
        }
        return matching;
    }
};
```

3.15 Hungarian Algorithm (Kuhn-Munkres)

Finds the maximum weight (or minimum cost) perfect matching in a complete bipartite graph in $O(V^3)$ time complexity.

```
/* Hash: A9711F63 */
const long long INF = 1e18;

struct Hungarian {
    int n;
    vector<vector<long long>> cost;
    vector<long long> u, v, dist;
    vector<int> p, way, match;
    vector<bool> used;

    Hungarian(int _n) : n(_n), cost(_n + 1, vector<long long>(_n +
      1, 0)), u(_n + 1, 0), v(_n + 1, 0), p(_n + 1, 0), way(_n + 1,
      0), match(_n + 1, 0), dist(_n + 1, 0), used(_n + 1, false) {}

    long long solve() {
        for (int i = 1; i <= n; ++i) {
            p[0] = i;
            int j_cur = 0;
            fill(dist.begin(), dist.end(), INF);
            fill(used.begin(), used.end(), false);
```

```
            do {
                used[j_cur] = true;
                int i_cur = p[j_cur], j_next = 0;
                long long delta = INF;
                for (int j = 1; j <= n; ++j) {
                    if (!used[j]) {
                        long long cur = cost[i_cur][j] - u[i_cur] -
     v[j];
                        if (cur < dist[j]) { dist[j] = cur; way[j]
    = j_cur; }
                        if (dist[j] < delta) { delta = dist[j];
    j_next = j; }
                    }
                }
                for (int j = 0; j <= n; ++j) {
                    if (used[j]) { u[p[j]] += delta; v[j] -= delta;
     }
                    else dist[j] -= delta;
                }
                j_cur = j_next;
            } while (p[j_cur] != 0);
            do {
                int j_prev = way[j_cur];
                p[j_cur] = p[j_prev];
                j_cur = j_prev;
            } while (j_cur != 0);
        }
        for (int j = 1; j <= n; ++j) match[p[j]] = j;
        return -v[0];
    }
};
```

- Initialize via `Hungarian hung(n)` (1-indexed). Populate `hung.cost[i][j]` with the cost of matching left node i with right node j.
- Call `hung.solve()` to find the minimum cost perfect matching. To find the maximum weight instead, negate all elements in the input cost matrix.
- The array `hung.match[i]` stores the index of the right node matched with left node i.

3.16 Kruskal Reconstruction Tree (KRT)

Max/min edge query on path becomes LCA query. Root of LCA gives `node_weight[LCA(u, v)]`. Preprocess LCA on `adj` up to `node_cnt` ($2N-1$ nodes).

```
/* Hash: DE683657 */
struct KRT {
    int n, node_cnt;
    vector<int> parent, sz, node_weight;
    vector<vector<int>> adj;

    KRT(int n) : n(n), node_cnt(n), parent(2 * n + 1), sz(2 * n +
     1, 1), node_weight(2 * n + 1, 0), adj(2 * n + 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find_set(int v) {
        return v == parent[v] ? v : parent[v] = find_set(parent[v])
     ;
    }

    void build(vector<tuple<int, int, int>>& edges) {
        sort(edges.begin(), edges.end());
        for (auto [w, u, v] : edges) {
            int root_u = find_set(u), root_v = find_set(v);
            if (root_u != root_v) {
                node_cnt++;
                node_weight[node_cnt] = w;
                parent[root_u] = node_cnt; parent[root_v] =
     node_cnt;
                adj[node_cnt].push_back(root_u); adj[node_cnt].
     push_back(root_v);
            }
        }
    }
};
```

3.17 Link-Cut Tree

A data structure that maintains a forest of rooted trees under dynamic edge insertions (`link`) and deletions (`cut`). All operations run in $O(\log N)$ amortized time using Splay Trees.

```
/* Hash: 6F0B1773 */
struct Node {
    int p = 0, ch[2] = {0, 0};
    bool rev = false;
};

vector<Node> t;

void init_lct(int n) {
    t.assign(n + 1, Node());
}

bool is_root(int x) {
    return t[t[x].p].ch[0] != x && t[t[x].p].ch[1] != x;
}

void push(int x) {
    if (x && t[x].rev) {
        swap(t[x].ch[0], t[x].ch[1]);
        if (t[x].ch[0]) t[t[x].ch[0]].rev ^= 1;
        if (t[x].ch[1]) t[t[x].ch[1]].rev ^= 1;
        t[x].rev = false;
    }
}

void push_all(int x) {
    if (!is_root(x)) push_all(t[x].p);
    push(x);
}

void rotate(int x) {
    int y = t[x].p, z = t[y].p;
    int k = (t[y].ch[1] == x);
    if (!is_root(y)) t[z].ch[t[z].ch[1] == y] = x;
    t[x].p = z;
    t[y].ch[k] = t[x].ch[k ^ 1];
    if (t[x].ch[k ^ 1]) t[t[x].ch[k ^ 1]].p = y;
    t[x].ch[k ^ 1] = y;
    t[y].p = x;
}

void splay(int x) {
    push_all(x);
    while (!is_root(x)) {
        int y = t[x].p, z = t[y].p;
        if (!is_root(y)) {
            if ((t[y].ch[1] == x) ^ (t[z].ch[1] == y)) rotate(x);
            else rotate(y);
        }
        rotate(x);
    }
}

void access(int x) {
    for (int t_ch = 0, r = x; r; t_ch = r, r = t[r].p) {
        splay(r);
        t[r].ch[1] = t_ch;
    }
    splay(x);
}

void make_root(int x) {
    access(x);
    t[x].rev ^= 1;
}

int find_root(int x) {
    access(x);
    while (t[x].ch[0]) {
        push(x);
        x = t[x].ch[0];
    }
    splay(x);
    return x;
}

void link(int x, int y) {
    make_root(x);
    if (find_root(y) != x) t[x].p = y;
}

void cut(int x, int y) {
    make_root(x);
    if (find_root(y) == x && t[y].p == x && !t[y].ch[0]) {
        t[y].p = t[x].ch[1] = 0;
    }
}
```

- Call `init_lct(n)` to allocate memory for n nodes (1-indexed).
- Call `link(x, y)` to add an undirected edge between x and y. It automatically checks connectivity to prevent cycles.
- Call `cut(x, y)` to remove the edge between x and y if it exists.
- Call `make_root(x)` to change the root of the tree containing x to be x.
- Call `find_root(x)` to get the actual component root of x. Useful for dynamic connectivity checks via `find_root(x) == find_root(y)`.

3.18 Boruvka's Algorithm

MST in $O(E \log V)$. Efficient when component-wise cheap edges can be found faster than scanning all edges. Returns -1 if graph is disconnected.

```
/* Hash: 07F7CC45 */
struct Boruvka {
    struct Edge { int u, v; long long weight; int id; };
    int n; vector<Edge> edges; vector<int> parent, sz;

    Boruvka(int n) : n(n), parent(n + 1), sz(n + 1, 1) { iota(
     parent.begin(), parent.parent.end(), 0); }

    void add_edge(int u, int v, long long w, int id = 0) { edges.
     push_back({u, v, w, id}); }
```

```
    int find_set(int v) { return v == parent[v] ? v : parent[v] =
     find_set(parent[v]); }

    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a; sz[a] += sz[b]; return true;
    }

    long long solve(vector<int>& mst_edges) {
        long long total_weight = 0; int num_components = n;
        vector<int> cheapest(n + 1);
        while (num_components > 1) {
            fill(cheapest.begin(), cheapest.end(), -1);
            for (int i = 0; i < edges.size(); i++) {
                int set_u = find_set(edges[i].u), set_v = find_set(
     edges[i].v);
                if (set_u == set_v) continue;
                if (cheapest[set_u] == -1 || edges[i].weight <
     edges[cheapest[set_u]].weight) cheapest[set_u] = i;
                if (cheapest[set_v] == -1 || edges[i].weight <
     edges[cheapest[set_v]].weight) cheapest[set_v] = i;
            }
            bool flag = false;
            for (int i = 1; i <= n; i++) if (cheapest[i] != -1) {
                int idx = cheapest[i];
                if (union_sets(edges[idx].u, edges[idx].v)) {
                    total_weight += edges[idx].weight; mst_edges.
     push_back(edges[idx].id);
                    num_components--; flag = true;
                }
            }
            if (!flag) break;
        }
        return (num_components == 1) ? total_weight : -1;
    }
};
```

3.19 Prim's Algorithm

MST in $O(E \log V)$. Grows a single tree component. Returns -1 if graph is disconnected. Populates `mst_edges` with selected edge IDs.

```
/* Hash: DA1A8008 */
const long long INF = 1e18;
struct Edge { int to, weight, id; };

long long prim(int source, int n, const vector<vector<Edge>>& adj,
     vector<int>& mst_edges) {
    long long total_weight = 0; int visited_count = 0;
    vector<bool> visited(n + 1, false);
    vector<long long> min_edge(n + 1, INF);
    vector<int> parent_edge(n + 1, -1);

    priority_queue<pair<long long, int>, vector<pair<long long, int
     >>, greater<pair<long long, int>>> pq;
    min_edge[source] = 0; pq.push({0, source});

    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (visited[u]) continue;

        visited[u] = true; total_weight += w; visited_count++;
        if (parent_edge[u] != -1) mst_edges.push_back(parent_edge[u
     ]);

        for (const auto& edge : adj[u]) {
            if (!visited[edge.to] && edge.weight < min_edge[edge.to
     ]) {
                min_edge[edge.to] = edge.weight; parent_edge[edge.
     to] = edge.id;
                pq.push({min_edge[edge.to], edge.to});
            }
        }
    }
    return (visited_count == n) ? total_weight : -1;
}
```

3.20 Tarjan Algorithm

Finds Strongly Connected Components (SCC) in directed graphs in $O(V + E)$. Components are labeled 1 to `scc_count` in `scc_id`.

```
/* Hash: EACC1A0F */
struct Tarjan {
    int n, timer, scc_count; vector<vector<int>> adj;
    vector<int> tin, low, scc_id; vector<bool> in_stack; stack<int>
      st;

    Tarjan(int n, const vector<vector<int>>& adj) :
        n(n), adj(adj), timer(0), scc_count(0), tin(n + 1, -1), low
     (n + 1, -1), scc_id(n + 1, -1), in_stack(n + 1, false) {}

    void dfs(int u) {
        tin[u] = low[u] = ++timer; st.push(u); in_stack[u] = true;
        for (int v : adj[u]) {
            if (tin[v] == -1) {
                dfs(v); low[u] = min(low[u], low[v]);
            } else if (in_stack[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }
        if (low[u] == tin[u]) {
            scc_count++;
            while (true) {
                int v = st.top(); st.pop(); in_stack[v] = false;
     scc_id[v] = scc_count;
                if (u == v) break;
            }
        }
    }

    void run() {
        for (int i = 1; i <= n; i++) if (tin[i] == -1) dfs(i);
    }
};
```

3.21 Virtual Tree (Auxiliary Tree)

Builds a compressed tree of K marked nodes and their LCAs in $O(K \log K)$. Clear `v_adj` via `k_nodes` after use.

```
/* Hash: 305AF4C7 */
struct VirtualTree {
    LCA &lca; vector<vector<int>> v_adj;
    VirtualTree(LCA &lca) : lca(lca), v_adj(lca.n + 1) {}

    void build(vector<int> &k_nodes) {
        auto cmp = [&](int a, int b) { return lca.tin[a] < lca.tin[
     b]; };
        sort(k_nodes.begin(), k_nodes.end(), cmp);

        int m = k_nodes.size();
        for (int i = 0; i < m - 1; i++) k_nodes.push_back(lca.query
     (k_nodes[i], k_nodes[i + 1]));
        sort(k_nodes.begin(), k_nodes.end(), cmp);
        k_nodes.erase(unique(k_nodes.begin(), k_nodes.end()),
     k_nodes.end());
        for (int u : k_nodes) v_adj[u].clear();
        vector<int> st;
        for (int u : k_nodes) {
            while (!st.empty() && !lca.is_ancestor(st.back(), u))
     st.pop_back();
            if (!st.empty()) v_adj[st.back()].push_back(u);
            st.push_back(u);
        }
    }
};
```

4 Math

4.1 Chinese Remainder Theorem (CRT)

Solves $x \equiv r_i \pmod{m_i}$ for non-coprime moduli. Returns `{ans, lcm}`, or `{-1, -1}` if impossible.

```
/* Hash: 459F49A8 */
long long extgcd(long long a, long long b, long long &x, long long
     &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, d = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b); return d;
}

pair<long long, long long> crt_two(long long r1, long long m1, long
      long r2, long long m2) {
    long long x, y, g = extgcd(m1, m2, x, y);
    if ((r2 - r1) % g != 0) return {-1, -1};
    long long lcm = m1 / g * m2, mod = m2 / g;
    long long x0 = ((x % mod) + mod) % mod;
    long long ans = (r1 + (__int128)((r2 - r1) / g) * x0 % lcm * m1
     ) % lcm;
    return {ans < 0 ? ans + lcm : ans, lcm};
}

pair<long long, long long> crt(const vector<pair<long long, long
     long>>& eq) {
    if (eq.empty()) return {0, 1};
    long long r_ans = eq[0].first, m_ans = eq[0].second;
    for (size_t i = 1; i < eq.size(); i++) {
        auto [r, m] = crt_two(r_ans, m_ans, eq[i].first, eq[i].
     second);
        if (m == -1) return {-1, -1};
        r_ans = r; m_ans = m;
    }
    return {r_ans, m_ans};
}
```

Usage guide for $x \equiv r_i \pmod{m_i}$:

- Pass equations as `{r_i, m_i}` into `vector<pair<long long, long long>> eq`.
- Call `auto [ans, lcm] = crt(eq);`. If `lcm == -1`, no solution exists.
- Safe guard: Ensure $0 \le r_i < m_i$ before calling to avoid incorrect `r2 - r1` signs inside.

4.2 Euler's Totient Function (Phi)

Single query in $O(\sqrt{N})$. Sieve precomputation up to N in $O(N \log \log N)$.

```cpp
/* Hash: 3653EA56 */
long long phi_single(long long n) {
    long long res = n;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}

vector<int> phi_sieve(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++) if (phi[i] == i) {
        for (int j = i; j <= n; j += i) phi[j] -= phi[j] / i;
    }
    return phi;
}
```

4.3 Extended Euclidean

Computes $\gcd(a, b)$ and finds x, y such that $ax + by = \gcd(a, b)$ in $O(\log(\min(a, b)))$. Modular inverse of $a \pmod m$ is `(x % m + m) % m` if $\gcd == 1$.

```cpp
/* Hash: 22A3BA61 */
long long extgcd(long long a, long long b, long long &x, long long
    &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, d = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b); return d;
}
```

4.4 FFT & NTT

Polynomial multiplication in $O(N \log N)$. Use `FFT` for high-precision floating-point coefficients and `NTT` for exact integer multiplication under a large prime modulus (typically `998244353`).

```cpp
/* Hash: 4F119655 */
const double PI = acos(-1);
struct Complex {
    double r, i;
    Complex(double r = 0, double i = 0) : r(r), i(i) {}
    Complex operator+(const Complex& o) const { return {r + o.r, i
     + o.i}; }
    Complex operator-(const Complex& o) const { return {r - o.r, i
     - o.i}; }
    Complex operator*(const Complex& o) const { return {r * o.r - i
      * o.i, r * o.i + i * o.r}; }
};

void fft(vector<Complex>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        Complex wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            Complex w(1);
            for (int j = 0; j < len / 2; j++) {
                Complex u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w = w * wlen;
            }
        }
    }
    if (invert) {
        for (auto& x : a) x.r /= n;
    }
}

vector<long long> multiply_fft(const vector<int>& a, const vector<
    int>& b) {
    vector<Complex> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1; while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] = fa[i] * fb[i];
    fft(fa, true);
    vector<long long> res(n);
    for (int i = 0; i < n; i++) res[i] = round(fa[i].r);
    return res;
}
```

```cpp
/* Hash: 611FD53F */
const int MOD = 998244353;
const int G = 3; // Primitive root for 998244353

long long power(long long base, long long p) {
    long long res = 1; base %= MOD;
    while (p > 0) {
        if (p & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        p >>= 1;
    }
    return res;
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long wlen = power(G, (MOD - 1) / len);
        if (invert) wlen = power(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; j++) {
                long long u = a[i + j], v = (a[i + j + len / 2] * w
    ) % MOD;
                a[i + j] = (u + v < MOD ? u + v : u + v - MOD);
                a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v +
    MOD);
                w = (w * wlen) % MOD;
            }
        }
    }
    if (invert) {
        long long n_inv = power(n, MOD - 2);
        for (auto& x : a) x = (x * n_inv) % MOD;
    }
}

vector<long long> multiply_ntt(const vector<long long>& a, const
    vector<long long>& b) {
    vector<long long> fa(a.begin(), a.end()), fb(b.begin(), b.end()
    );
    int n = 1; while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n, 0); fb.resize(n, 0);
    ntt(fa, false); ntt(fb, false);
    for (int i = 0; i < n; i++) fa[i] = (fa[i] * fb[i]) % MOD;
    ntt(fa, true);
    return fa;
}
```

To multiply two polynomials $A(x)$ and $B(x)$, pass their coefficients to `multiply_fft` or `multiply_ntt`. For example:

```cpp
/* Hash: 8CBF2CBC */
// Multiply (1 + 2x) * (2 + 3x) -> 2 + 7x + 6x^2
vector<int> a = {1, 2};
vector<int> b = {2, 3};
vector<long long> c = multiply_fft(a, b);
// c contains: {2, 7, 6, 0, ...}
```

4.5 Matrix Exponentiation Template

Solves linear recurrence relations and DP optimizations in $O(K^3 \log N)$. Handles modular addition, subtraction, multiplication, and fast power.

```cpp
/* Hash: DDB4A8A9 */
const int MOD = 1e9 + 7;

struct Matrix {
    int r, c; vector<vector<long long>> mat;

    Matrix(int r, int c, bool identity = false) : r(r), c(c), mat(r
     , vector<long long>(c, 0)) {
        if (identity) for (int i = 0; i < min(r, c); i++) mat[i][i]
     = 1;
    }

    Matrix operator+(const Matrix& o) const {
        Matrix res(r, c);
        for (int i = 0; i < r; i++) for (int j = 0; j < c; j++)
            res.mat[i][j] = (mat[i][j] + o.mat[i][j]) % MOD;
        return res;
    }

    Matrix operator-(const Matrix& o) const {
        Matrix res(r, c);
        for (int i = 0; i < r; i++) for (int j = 0; j < c; j++)
            res.mat[i][j] = (mat[i][j] - o.mat[i][j] + MOD) % MOD;
        return res;
    }

    Matrix operator*(const Matrix& o) const {
        Matrix res(r, o.c);
        for (int i = 0; i < r; i++) for (int k = 0; k < c; k++) for
      (int j = 0; j < o.c; j++)
            res.mat[i][j] = (res.mat[i][j] + mat[i][k] * o.mat[k][j
     ]) % MOD;
        return res;
    }

    Matrix power(long long p) const {
        Matrix res(r, r, true), base = *this;
        while (p > 0) {
            if (p & 1) res = res * base;
            base = base * base; p >>= 1;
        }
        return res;
    }
};
```

Fibonacci F_n example ($T^{n-1} \cdot F_1$):

```
/* Hash: 69D60AEF */
Matrix T(2, 2); T.mat = {{1, 1}, {1, 0}};
Matrix F(2, 1); F.mat = {{1}, {0}}; // F[1]=1, F[0]=0

Matrix Ans = T.power(n - 1) * F;
long long fn = Ans.mat[0][0];
```

4.6 Miller-Rabin Primality Test & Pollard's Rho

Deterministic 64-bit primality check in $O(\log N)$ and factorization in $O(N^{1/4})$.

```
/* Hash: F2DD86E5 */
typedef unsigned long long ull;

ull mul_mod(ull a, ull b, ull m) {
    long long res = a * b - m * (ull)((long double)a * b / m);
    return (res < 0 ? res + m : res) % m;
}

ull power(ull base, ull p, ull mod) {
    ull res = 1; base %= mod;
    while (p > 0) {
        if (p & 1) res = mul_mod(res, base, mod);
        base = mul_mod(base, base, mod); p >>= 1;
    }
    return res;
}

bool miller_rabin(ull n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;
    ull d = n - 1; int s = 0;
    while (d % 2 == 0) { d /= 2; s++; }
    static const ull bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
      31, 37};
    for (ull a : bases) {
        if (n <= a) break;
        ull x = power(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (int r = 1; r < s; r++) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}

ull pollard_rho(ull n) {
    if (n % 2 == 0) return 2;
    if (miller_rabin(n)) return n;
    ull x = 2, y = 2, d = 1, c = 1;
    auto f = [&](ull x, ull n, ull c) { return (mul_mod(x, x, n) +
     c) % n; };
    while (d == 1) {
        x = f(x, n, c); y = f(f(y, n, c), n, c);
        d = std::gcd(x > y ? x - y : y - x, n);
        if (d == n) { x = rand() % (n - 2) + 2; y = x; c = rand() %
      (n - 1) + 1; d = 1; }
    }
    return d;
}

void factorize(ull n, vector<ull>& factors) {
    if (n == 1) return;
    if (miller_rabin(n)) { factors.push_back(n); return; }
    ull p = pollard_rho(n); factorize(p, factors); factorize(n / p,
      factors);
}
```

4.7 Stars and Bars (Chia kẹo Euler)

Finds the number of integer solutions to equations and inequalities.

1. Standard ($x_i \geq 0$):

$$\binom{n+k-1}{k-1}$$

2. Positive integers ($x_i \geq 1$):

$$\binom{n-1}{k-1}$$

3. Lower bounds ($x_i \geq c_i$): Let $n' = n - \sum c_i$.

$$\binom{n'+k-1}{k-1}$$

4. Identical upper bounds ($0 \leq x_i \leq L$):

$$\sum_{i=0}^{k} (-1)^i \binom{k}{i} \binom{n - i(L+1) + k - 1}{k-1}$$

5. Distinct upper bounds ($0 \leq x_i \leq L_i$): Let S be a subset of indices $\{1, 2, \ldots, k\}$.

$$\sum_{S} (-1)^{|S|} \binom{n - \sum_{j \in S}(L_j + 1) + k - 1}{k-1}$$

6. Non-negative Inequality ($\sum_{i=1}^{k} x_i \leq n$ where $x_i \geq 0$):

$$\binom{n+k}{k}$$

7. Positive Inequality ($\sum_{i=1}^{k} x_i \leq n$ where $x_i \geq 1$):

$$\binom{n-1+k}{k}$$

Note: $\binom{n}{k} = 0$ if $n < k$ or $k < 0$.

4.8 Binary GCD (Stein's Algorithm)

```
/* Hash: 840D562E */
long long fast_gcd(long long a, long long b) {
    if (!a || !b) return a ^ b;
    int shift = __builtin_ctzll(a | b);
    a >>= __builtin_ctzll(a);
    while (b) {
        b >>= __builtin_ctzll(b);
        if (a > b) swap(a, b);
        b -= a;
    }
    return a << shift;
}
```

4.9 Stern-Brocot Tree

A data structure that generates all positive irreducible fractions exactly once by maintaining lower and upper bounds.

```
/* Hash: 2F718C99 */
struct Fraction {
    long long p, q;
    bool operator==(const Fraction& o) const { return p * o.q == q
     * o.p; }
    bool operator<(const Fraction& o) const { return p * o.q < q *
     o.p; }
};

struct SternBrocot {
    Fraction mediant(Fraction l, Fraction r) {
        return {l.p + r.p, l.q + r.q};
    }

    string find_path(Fraction target, int max_depth = 60) {
        Fraction L = {0, 1}, R = {1, 0};
        string path = "";
        for (int i = 0; i < max_depth; ++i) {
            Fraction M = mediant(L, R);
            if (target == M) break;
            if (target < M) {
                path += 'L';
                R = M;
            } else {
                path += 'R';
                L = M;
            }
        }
        return path;
    }
};
```

- Call `mediant(l, r)` to get the fraction between bounds l and r.
- Call `find_path(target)` to trace the path from the root to the `target`.
- Every generated fraction satisfies $\gcd(p, q) = 1$ automatically.

4.10 Tonelli-Shanks Algorithm

Solves the modular congruence $x^2 \equiv n \pmod{p}$ for a prime p in $O(\log^2 p)$ time complexity. It finds the modular square root if it exists.

```
/* Hash: 36C825BE */
long long power(long long base, long long p, long long mod) {
    long long res = 1; base %= mod;
    while (p > 0) {
        if (p & 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod;
        p >>= 1;
    }
    return res;
}

long long tonelli_shanks(long long n, long long p) {
    if (p == 2) return n & 1;
    if (power(n, (p - 1) / 2, p) != 1) return -1; // No solution (
     Legendre symbol)
    if (p % 4 == 3) return power(n, (p + 1) / 4, p);

    long long q = p - 1, s = 0;
    while (q % 2 == 0) { q /= 2; s++; }

    long long z = 2;
    while (power(z, (p - 1) / 2, p) == 1) z++;

    long long m = s, c = power(z, q, p), t = power(n, q, p), r =
     power(n, (q + 1) / 2, p);

    while (t != 1) {
        long long t2k = t, k = 0;
        for (k = 1; k < m; k++) {
            t2k = (__int128)t2k * t2k % p;
            if (t2k == 1) break;
        }
        if (k == m) return -1;
        long long b = power(c, 1LL << (m - k - 1), p);
        r = (__int128)r * b % p;
        c = (__int128)b * b % p;
        t = (__int128)t * c % p;
        m = k;
    }
    return r;
}
```

Call `tonelli_shanks(n, p)` to find a value x such that $x^2 \equiv n \pmod{p}$.

- Returns a valid solution $x \in [0, p-1]$ if it exists.
- Returns -1 if n is a quadratic non-residue modulo p (no solution exists).
- Note that if x is a solution, $p - x$ is also a valid solution.

5 Strings

5.1 Aho-Corasick Automaton

Multi-pattern matching in $O(\sum |pattern| + |text|)$. Converts Trie to DFA via transition table optimization.

```
/* Hash: 9E3E226E */
struct AhoCorasick {
    static const int ALPHABET_SIZE = 26;
    struct Node {
        int next[ALPHABET_SIZE], link = -1, exit_link = -1;
        vector<int> ids;
        Node() { memset(next, -1, sizeof(next)); }
    };

    vector<Node> trie;
    AhoCorasick() { trie.push_back(Node()); }

    void insert(const string& s, int id) {
        int u = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (trie[u].next[idx] == -1) {
                trie[u].next[idx] = trie.size(); trie.push_back(
     Node());
            }
            u = trie[u].next[idx];
        }
        trie[u].ids.push_back(id);
    }

    void build() {
        queue<int> q;
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            if (trie[0].next[i] != -1) { trie[trie[0].next[i]].link
      = 0; q.push(trie[0].next[i]); }
            else trie[0].next[i] = 0;
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int i = 0; i < ALPHABET_SIZE; i++) {
                int v = trie[u].next[i];
                if (v != -1) {
                    trie[v].link = trie[trie[u].link].next[i];
                    int slink = trie[v].link;
                    trie[v].exit_link = !trie[slink].ids.empty() ?
     slink : trie[slink].exit_link;
                    q.push(v);
                } else {
                    trie[u].next[i] = trie[trie[u].link].next[i];
                }
            }
        }
    }
};
```

5.2 KMP Algorithm

```
/* Hash: E495838F */
vector<int> prefix_function(string s) {
    int n = s.length(); vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_search(string text, string pattern) {
    string s = pattern + "#" + text;
    int n = s.length(), m = pattern.length();
    vector<int> pi = prefix_function(s), matches;
    for (int i = m + 1; i < n; i++) {
        if (pi[i] == m) matches.push_back(i - 2 * m);
    }
    return matches;
}
```

5.3 Manacher's Algorithm

Computes palindromic radii for all positions in $O(N)$. Max palindrome length at i is $p[i]$.

```
/* Hash: 98C37AC1 */
vector<int> manacher(string s) {
    string t = "#";
    for (char c : s) t += c, t += "#";
    int n = t.length(); vector<int> p(n, 0);
    int c = 0, r = 0;

    for (int i = 0; i < n; i++) {
        int mirror = 2 * c - i;
        if (i < r) p[i] = min(r - i, p[mirror]);
        while (i + p[i] + 1 < n && i - p[i] - 1 >= 0 && t[i + p[i]
     + 1] == t[i - p[i] - 1]) p[i]++;
        if (i + p[i] > r) { c = i; r = i + p[i]; }
    }
    return p;
}
```

5.4 Palindrome Tree (Eertree)

A specialized tree structure that processes characters online to maintain all unique palindromic substrings of a string in $O(N)$ time and space complexity.

```
/* Hash: FD7BD484 */
struct PalindromeTree {
    static const int ALPHABET_SIZE = 26;
    struct Node {
        int next[ALPHABET_SIZE];
        int len;
        int link;
        int cnt; // Frequency of the palindrome substring
    };

    string s;
    vector<Node> tree;
    int sz, last;

    PalindromeTree() {
        s = "";
        // Node 1: Imaginary root of odd length (-1)
        // Node 2: Root of even length (0)
        tree.push_back({}); // Dummy at index 0
        tree.push_back({}); tree[1].len = -1; tree[1].link = 1;
        tree.push_back({}); tree[2].len = 0;  tree[2].link = 1;
        memset(tree[1].next, 0, sizeof(tree[1].next));
        memset(tree[2].next, 0, sizeof(tree[2].next));
        sz = 2; last = 2;
    }

    int get_link(int u) {
        int curr_pos = s.length() - 1;
        while (true) {
            int prev_pos = curr_pos - 1 - tree[u].len;
            if (prev_pos >= 0 && s[prev_pos] == s[curr_pos]) return
     u;
            u = tree[u].link;
        }
    }

    void insert(char ch) {
        s += ch;
        int c = ch - 'a';
        int curr = get_link(last);

        if (tree[curr].next[c]) {
            last = tree[curr].next[c];
```

```
            tree[last].cnt++;
            return;
        }

        sz++;
        last = sz;
        tree.push_back({});
        memset(tree[sz].next, 0, sizeof(tree[sz].next));

        tree[sz].len = tree[curr].len + 2;
        tree[curr].next[c] = sz;

        if (tree[sz].len == 1) {
            tree[sz].link = 2;
            tree[sz].cnt = 1;
            return;
        }

        int link_node = get_link(tree[curr].link);
        tree[sz].link = tree[link_node].next[c];
        tree[sz].cnt = 1;
    }

    void pull_frequencies() {
        // Propagate counts from longer palindromes to their suffix
       links
        for (int i = sz; i > 2; i--) {
            tree[tree[i].link].cnt += tree[i].cnt;
        }
    }
};
```

Initialize via `PalindromeTree pt` and append characters one by one using `pt.insert(c)`. Call `pt.pull_frequencies()` at the end to get the exact total occurrence count of every unique palindromic substring inside `tree[i].cnt`.

5.5 String Double Hashing

Anti-hash-kill rolling hash with random bases. Input range $[l, r]$ for `get(l, r)` is 0-indexed.

```
/* Hash: 27B9BF00 */
struct StringHash {
    long long MOD1 = 1e9 + 7, MOD2 = 1e9 + 9, BASE1, BASE2;
    vector<long long> h1, h2, p1, p2;

    StringHash(const string& s) {
        BASE1 = (rng() % (MOD1 - 300) + 300) | 1;
        BASE2 = (rng() % (MOD2 - 300) + 300) | 1;
        int n = s.size();
        h1.resize(n + 1, 0); h2.resize(n + 1, 0);
        p1.resize(n + 1, 1); p2.resize(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * BASE1 + s[i]) % MOD1;
            h2[i + 1] = (h2[i] * BASE2 + s[i]) % MOD2;
            p1[i + 1] = (p1[i] * BASE1) % MOD1;
            p2[i + 1] = (p2[i] * BASE2) % MOD2;
        }
    }

    pair<long long, long long> get(int l, int r) {
        l++; r++;
        long long res1 = (h1[r] - h1[l - 1] * p1[r - l + 1]) % MOD1
     ;
        long long res2 = (h2[r] - h2[l - 1] * p2[r - l + 1]) % MOD2
     ;
        return {res1 < 0 ? res1 + MOD1 : res1, res2 < 0 ? res2 +
     MOD2 : res2};
    }
};
```

5.6 Suffix Array

Constructs SA and LCP in $O(N \log N)$. Appends sentinel `'$'` internally. `sa[1]` is the first real suffix, `lcp[i]` stores LCP between `sa[i]` and `sa[i-1]`.

```
/* Hash: 467FBF4B */
struct SuffixArray {
    string s; int n; vector<int> sa, rank, lcp;
    SuffixArray(string s) : s(s + "$"), n(s.length() + 1) { sa.
     resize(n); rank.resize(n); build_sa(); build_lcp(); }

    void build_sa() {
        vector<int> p(n), c(n), cnt(max(256, n), 0), pn(n), cn(n);
        for (int i = 0; i < n; i++) cnt[s[i]]++;
        for (int i = 1; i < 256; i++) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
        c[p[0]] = 0; int classes = 1;
        for (int i = 1; i < n; i++) {
            if (s[p[i]] != s[p[i - 1]]) classes++;
            c[p[i]] = classes - 1;
        }
        for (int h = 0; (1 << h) < n; ++h) {
            for (int i = 0; i < n; i++) pn[i] = (p[i] - (1 << h) +
     n) % n;
            fill(cnt.begin(), cnt.begin() + classes, 0);
            for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] =
     pn[i];
            cn[p[0]] = 0; classes = 1;
            for (int i = 1; i < n; i++) {
                if (c[p[i]] != c[p[i - 1]] || c[(p[i] + (1 << h)) %
      n] != c[(p[i - 1] + (1 << h)) % n]) classes++;
                cn[p[i]] = classes - 1;
            }
            c = cn;
        }
        sa = p; for (int i = 0; i < n; i++) rank[sa[i]] = i;
    }

    void build_lcp() {
        lcp.assign(n, 0); int k = 0;
        for (int i = 0; i < n - 1; i++) {
            int pi = rank[i], j = sa[pi - 1];
            while (s[i + k] == s[j + k]) k++;
            lcp[pi] = k; if (k) k--;
        }
    }
};
```

5.7 Suffix Automaton (SAM)

A directed acyclic word graph that represents all suffixes of a string in $O(N)$ time and space. Each state represents a set of substrings that share the same end-positions (`endpos`).

```
/* Hash: B920F058 */
struct State {
    int len, link;
    int next[26];
};

const int MAXLEN = 100005;
State st[MAXLEN * 2];
int sz, last;

void sam_init() {
    st[0].len = 0;
    st[0].link = -1;
    memset(st[0].next, -1, sizeof(st[0].next));
    sz = 1;
    last = 0;
}

void sam_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    memset(st[cur].next, -1, sizeof(st[cur].next));

    int p = last;
    while (p != -1 && st[p].next[c - 'a'] == -1) {
        st[p].next[c - 'a'] = cur;
        p = st[p].link;
    }

    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c - 'a'];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            memcpy(st[clone].next, st[q].next, sizeof(st[q].next));
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c - 'a'] == q) {
                st[p].next[c - 'a'] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
```

- Call `sam_init()` once before building the automaton.
- Call `sam_extend(c)` sequentially for each character in the string to construct the SAM online.
- The total number of states in the automaton is at most $2N - 1$. Ensure `MAXLEN` is at least the maximum string length.
- Useful for complex string operations such as substring counting, longest common substring of multiple strings, and finding the lexicographically k-th substring.

5.8 Trie

Array-based Trie acting as a multiset. Allows duplicate words, $O(L)$ insertion, search, and deletion.

```
/* Hash: CF416890 */
struct Trie {
    static const int ALPHABET_SIZE = 26;
    struct Node {
        int next[ALPHABET_SIZE], cnt = 0, is_end = 0;
        Node() { memset(next, -1, sizeof(next)); }
    };

    vector<Node> trie;
    Trie() { trie.push_back(Node()); }

    void insert(const string& s) {
        int u = 0; trie[u].cnt++;
```

```
        for (char c : s) {
            int idx = c - 'a';
            if (trie[u].next[idx] == -1) {
                trie[u].next[idx] = trie.size(); trie.push_back(
    Node());
            }
            u = trie[u].next[idx]; trie[u].cnt++;
        }
        trie[u].is_end++;
    }

    bool search(const string& s) {
        int u = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (trie[u].next[idx] == -1 || trie[trie[u].next[idx]].
    cnt == 0) return false;
            u = trie[u].next[idx];
        }
        return trie[u].is_end > 0;
    }

    void remove(const string& s) {
        if (!search(s)) return;
        int u = 0; trie[u].cnt--;
        for (char c : s) {
            int idx = c - 'a', nxt = trie[u].next[idx];
            trie[nxt].cnt--; u = nxt;
        }
        trie[u].is_end--;
    }
};
```

5.9 Z-Algorithm

Computes Z-array in $O(N)$ where $Z[i]$ is the LCP length of s and its suffix starting at i.

```
/* Hash: 0521963F */
vector<int> z_function(string s) {
    int n = s.length(); vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

vector<int> z_search(string text, string pattern) {
    string s = pattern + "#" + text; int m = pattern.length();
    vector<int> z = z_function(s), matches;
    for (int i = m + 1; i < s.length(); i++) {
        if (z[i] == m) matches.push_back(i - m - 1);
    }
    return matches;
}
```

6 Bit Manipulation

6.1 Binary Trie (Bit Trie)

$O(B)$ max XOR query on 32-bit integers. Acting as a multiset using node counters.

```
/* Hash: 78F2249B */
struct BitTrie {
    static const int BITS = 30;
    struct Node {
        int next[2] = {0, 0}, cnt = 0;
    };
    vector<Node> trie;
    BitTrie() { trie.push_back(Node()); }

    void insert(int x) {
        int u = 0; trie[u].cnt++;
        for (int i = BITS; i >= 0; i--) {
            int bit = (x >> i) & 1;
            if (!trie[u].next[bit]) { trie[u].next[bit] = trie.size
     (); trie.push_back(Node()); }
            u = trie[u].next[bit]; trie[u].cnt++;
        }
    }

    void remove(int x) {
        int u = 0; trie[u].cnt--;
        for (int i = BITS; i >= 0; i--) {
            u = trie[u].next[(x >> i) & 1]; trie[u].cnt--;
        }
    }

    int max_xor(int x) {
        int u = 0, res = 0;
        for (int i = BITS; i >= 0; i--) {
            int bit = (x >> i) & 1, toggle = bit ^ 1;
            if (trie[u].next[toggle] && trie[trie[u].next[toggle]].
     cnt > 0) {
                res |= (1 << i); u = trie[u].next[toggle];
            } else {
                u = trie[u].next[bit];
            }
        }
        return res;
    }
};
```

6.2 Bitwise Hacks

High-performance bitwise manipulations. $O(1)$ submask traversal and hardware-accelerated queries.

```
/* Hash: 433527C7 */
// 1. Core Bit Operations
x | (1LL << i) // Set i-th bit to 1
x & ~(1LL << i) // Set i-th bit to 0
x ^ (1LL << i) // Toggle i-th bit
(x >> i) & 1LL // Check if i-th bit is 1
long long lsb = x & -x; // Isolate lowest set bit (Fenwick)
long long clear_lsb = x & (x - 1); // Clear lowest set bit

// 2. Bulk & Range Bit Manipulations (k bits, Range [l, r])
(1LL << k) - 1 // Mask of k lowest bits (000...111)
x ^ ((1LL << k) - 1) // Toggle k lowest bits of x
((1LL << (r - l + 1)) - 1) << l // Mask for range [l, r] (inclusive
    )
x ^ (((1LL << (r - l + 1)) - 1) << l) // Toggle all bits in range [
    l, r]
x | (x + 1) // Set the lowest unset (0) bit to 1

// 3. Submask Traversal
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {} // Iterate
     all submasks

// 4. Built-ins & Helper Functions
__builtin_popcountll(x) // Count set bits
__builtin_ctzll(x) // Count trailing zeros (x > 0)
__builtin_clzll(x) // Count leading zeros (x > 0)
__lg(x) // Fast Floor(log2(x)) -> Sparse Table query
long long next_p2(long long x) { return x <= 1 ? 1 : 1LL << (64 -
    __builtin_clzll(x - 1)); } // Next power of 2
long long safe_mod_mul(unsigned long long a, unsigned long long b,
    unsigned long long m) { // O(1) mul modulo without overflow
    long long res; return __builtin_mul_overflow(a, b, &res) ? (
    long long)(((unsigned __int128)a * b) % m) : res % m;
}
```

6.3 XOR Basis

Vector space for maximum XOR subset queries in $O(B)$ time.

```
/* Hash: E66FDB72 */
struct XorBasis {
    static const int BITS = 60;
    long long basis[BITS]; int sz = 0;
    XorBasis() { memset(basis, 0, sizeof(basis)); }

    bool insert(long long mask) {
        for (int i = BITS - 1; i >= 0; i--) if ((mask >> i) & 1) {
            if (!basis[i]) { basis[i] = mask; sz++; return true; }
            mask ^= basis[i];
        }
        return false;
    }

    long long max_xor() {
        long long res = 0;
        for (int i = BITS - 1; i >= 0; i--) if ((res ^ basis[i]) >
    res) res ^= basis[i];
        return res;
    }
};
```

7 DPs

7.1 Aliens Trick (WQS Binary Search)

Convex/concave function DP optimization from $O(N \cdot K)$ to $O(N \log(\text{Max } C))$. Binary search penalty `lambda` for choosing exactly K items.

```
/* Hash: 0B1062C9 */
pair<long long, int> check(long long lambda) {
    vector<pair<long long, int>> dp(n + 1, {1e18, 0}); dp[0] = {0,
     0};
    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];
        long long cost = dp[i - 1].first + a[i] - lambda;
        if (cost < dp[i].first) dp[i] = {cost, dp[i - 1].second +
    1};
    }
    return dp[n];
}

long long solve_aliens() {
    long long l = 0, r = 1e12, ans = -1;
    while (l <= r) {
        long long mid = (l + r) >> 1; auto [cost, count] = check(
    mid);
        if (count >= k) { ans = cost + k * mid; r = mid - 1; }
        else l = mid + 1;
    }
    return ans;
}
```

7.2 Bitmask DP for Set Partitioning

Solves the optimal grouping problem (CSES 1653 "Elevator Rides") for N items with a maximum capacity X per group in

$O(N \cdot 2^N)$ time complexity.

```
/* Hash: A7C379B3 */
pair<int, long long> solve_elevator(int n, long long max_weight,
     const vector<long long>& w) {
    vector<pair<int, long long>> dp(1 << n);
    dp[0] = {1, 0};

    for (int mask = 1; mask < (1 << n); ++mask) {
        dp[mask] = {n + 1, 0};
        for (int i = 0; i < n; ++i) {
            if (mask & (1 << i)) {
                auto prev = dp[mask ^ (1 << i)];
                if (prev.second + w[i] <= max_weight) {
                    prev.second += w[i];
                } else {
                    prev.first++;
                    prev.second = w[i];
                }
                dp[mask] = min(dp[mask], prev);
            }
        }
    }
    return dp[(1 << n) - 1];
}
```

- `dp[mask]` stores a `pair` representing the minimum number of groups and the weight of the last group for a subset of items.
- Pairs are compared lexicographically: it minimizes the number of groups first, then minimizes the current group's weight.

7.3 Digit DP

```
/* Hash: 114B62D3 */
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

string S;
ll memo[20][2][2]; // pos, smaller, lead

ll dp(int pos, bool smaller, bool lead) {
    if (pos == S.length()) return 1;
    ll &ans = memo[pos][smaller][lead];
    if (ans != -1) return ans;

    ll res = 0;
    int lim = smaller ? 9 : (S[pos] - '0');

    for (int d = 0; d <= lim; d++) {
        // Add conditions here (e.g., using d and lead)
        res += dp(pos + 1, smaller || (d < lim), lead && (d == 0));
    }
    return ans = res;
}

ll solve(ll X) {
    if (X < 0) return 0;
    S = to_string(X);
    memset(memo, -1, sizeof(memo));
    return dp(0, false, true);
}

int main() {
    ll L, R;
    if (cin >> L >> R) cout << solve(R) - solve(L - 1) << "\n";
}
```

7.4 Divide & Conquer Optimization

Reduces DP from $O(N^2)$ to $O(N \log N)$ when transition satisfies monotonicity: $opt[i][j] \le opt[i][j+1]$.

```
/* Hash: 1763E421 */
long long cost(int k, int j) { return 0; }

void compute(int l, int r, int opt_l, int opt_r) {
    if (l > r) return;
    int mid = (l + r) >> 1, best_k = -1; dp_curr[mid] = 1e18;

    for (int k = opt_l; k <= min(mid, opt_r); k++) {
        long long current_cost = dp_before[k] + cost(k, mid);
        if (current_cost < dp_curr[mid]) { dp_curr[mid] =
     current_cost; best_k = k; }
    }
    compute(l, mid - 1, opt_l, best_k); compute(mid + 1, r, best_k,
      opt_r);
}

void solve_dp() {
    dp_before.assign(n + 1, 0); dp_curr.assign(n + 1, 0);
    for (int j = 1; j <= n; j++) dp_before[j] = cost(0, j);
    for (int i = 2; i <= g; i++) { compute(1, n, 0, n); dp_before =
      dp_curr; }
}
```

7.5 Knuth Optimization

Reduces DP from $O(N^3)$ to $O(N^2)$ when $opt[i][j-1] \le opt[i][j] \le opt[i+1][j]$. Cost function must satisfy quadrangle inequality and monotonicity.

```
/* Hash: 492ADDB4 */
long long cost(int i, int j) { return 0; }

void solve_dp() {
    dp.assign(n + 1, vector<long long>(n + 1, 0)); opt.assign(n +
     1, vector<int>(n + 1, 0));
    for (int i = 1; i <= n; i++) opt[i][i] = i;

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1; dp[i][j] = 1e18;
            for (int k = opt[i][j - 1]; k <= min(j - 1, opt[i + 1][
     j]); k++) {
                long long cur = dp[i][k] + dp[k + 1][j] + cost(i, j
     );
                if (cur < dp[i][j]) { dp[i][j] = cur; opt[i][j] = k
     ; }
            }
        }
    }
}
```

7.6 Sum Over Subsets DP (SOS DP)

Calculates $F(mask) = \sum_{sub \subseteq mask} A(sub)$ for all masks in $O(N \cdot 2^N)$ time complexity.

```
/* Hash: F08ABD7E */
void solve_sos(int n, vector<int>& a) {
    vector<int> dp = a; // dp[mask] initially stores A[mask]
    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < (1 << n); ++mask) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}
```

To compute supersets instead ($F(mask) = \sum_{mask \subseteq super} A(super)$), change the condition to check if the bit is NOT set: `if (!(mask & (1 << i)))` and transition via `dp[mask] += dp[mask ^ (1 << i)]`.

8 Geometry

8.1 Geometry Templates (Integer & Double)

Complete 2D geometry templates. Integer version for 100% precision; double version handles fractions using `EPS`.

```
/* Hash: FB4DD0B5 */
struct Point {
    long long x, y;
    Point(long long x = 0, long long y = 0) : x(x), y(y) {}
    Point operator+(const Point& o) const { return {x + o.x, y + o.
     y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.
     y}; }
};

long long dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
long long cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
long long dist2(Point a, Point b) { return dot(a - b, a - b); }

int ccw(Point a, Point b, Point a_c) {
    long long cr = cross(b - a, a_c - a);
    return cr == 0 ? 0 : (cr > 0 ? 1 : -1);
}

bool on_segment(Point p, Point a, Point b) {
    return ccw(a, b, p) == 0 && dot(a - p, b - p) <= 0;
}

bool intersect(Point a, Point b, Point a_c, Point d) {
    if (on_segment(a_c, a, b) || on_segment(d, a, b) || on_segment(
     a, a_c, d) || on_segment(b, a_c, d)) return true;
    return ccw(a, b, a_c) * ccw(a, b, d) < 0 && ccw(a_c, d, a) *
     ccw(a_c, d, b) < 0;
}
```

```
/* Hash: C71F20A6 */
const double EPS = 1e-9;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator+(const Point& o) const { return {x + o.x, y + o.
     y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.
     y}; }
    Point operator*(double c) const { return {x * c, y * c}; }
    Point operator/(double c) const { return {x / c, y / c}; }
};

double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
double dist2(Point a, Point b) { return dot(a - b, a - b); }
double dist(Point a, Point b) { return sqrt(dist2(a, b)); }

int ccw(Point a, Point b, Point a_c) {
    double cr = cross(b - a, a_c - a);
    return abs(cr) < EPS ? 0 : (cr > 0 ? 1 : -1);
```

```
}

bool on_segment(Point p, Point a, Point b) {
    return ccw(a, b, p) == 0 && dot(a - p, b - p) <= EPS;
}

bool intersect(Point a, Point b, Point a_c, Point d) {
    if (on_segment(a_c, a, b) || on_segment(d, a, b) || on_segment(
     a, a_c, d) || on_segment(b, a_c, d)) return true;
    return ccw(a, b, a_c) * ccw(a, b, d) < 0 && ccw(a_c, d, a) *
     ccw(a_c, d, b) < 0;
}

Point line_intersection(Point a, Point b, Point a_c, Point d) {
    double cp = cross(b - a, d - a_c); assert(abs(cp) > EPS);
    return a + (b - a) * (cross(a_c - a, d - a_c) / cp);
}
```

8.2 Convex Hull (Andrew's Monotone Chain)

Computes 2D Convex Hull in $O(N \log N)$. Returns points in CCW order. Change `<= 0` to `< 0` to include collinear points.

```
/* Hash: B7B63F1B */
long long cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

vector<Point> convex_hull(vector<Point>& pts) {
    int n = pts.size(), k = 0; if (n <= 3) return pts;
    vector<Point> hull(2 * n); sort(pts.begin(), pts.end());

    for (int i = 0; i < n; ++i) {
        while (k >= 2 && cross(hull[k - 2], hull[k - 1], pts[i]) <=
     0) k--;
        hull[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(hull[k - 2], hull[k - 1], pts[i]) <=
     0) k--;
        hull[k++] = pts[i];
    }
    hull.resize(k - 1); return hull;
}
```

8.3 Incenter & Circumcenter of a Triangle

Computes the incenter and circumcenter of a triangle. Ensure points are not collinear.

```
/* Hash: FCFBD567 */
Point get_incenter(Point A, Point B, Point C) {
    double a = dist(B, C), b = dist(A, C), c = dist(A, B), p = a +
     b + c;
    assert(p > 1e-9); return (A * a + B * b + C * c) / p;
}

Point get_circumcenter(Point A, Point B, Point C) {
    double d = 2 * (A.x * (B.y - C.y) + B.x * (C.y - A.y) + C.x * (
     A.y - B.y)); assert(abs(d) > 1e-9);
    double ax = A.x * A.x + A.y * A.y, bx = B.x * B.x + B.y * B.y,
     cx = C.x * C.x + C.y * C.y;
    double ux = (ax * (B.y - C.y) + bx * (C.y - A.y) + cx * (A.y -
     B.y)) / d;
    double uy = (ax * (C.x - B.x) + bx * (A.x - C.x) + cx * (B.x -
     A.x)) / d;
    return {ux, uy};
}
```

8.4 Li Chao Tree

Maximum query of lines on $[L, R]$ in $O(\log(R - L))$. For Minimum: init with `{0, INF}`, change comparison to `<`, and use `min()` in query.

```
/* Hash: 73B73917 */
struct Line {
    long long k, m;
    long long operator()(long long x) { return k * x + m; }
};

struct LiChaoTree {
    int n; vector<Line> tree;
    LiChaoTree(int n) : n(n), tree(4 * n, {0, -INF}) {}

    void insert(int node, int l, int r, Line newline) {
        int mid = (l + r) >> 1;
        bool bit_l = newline(l) > tree[node](l), bit_m = newline(
     mid) > tree[node](mid);
        if (bit_m) swap(tree[node], newline);
        if (l == r) return;
        if (bit_l != bit_m) insert(node << 1, l, mid, newline);
        else insert(node << 1 | 1, mid + 1, r, newline);
    }

    long long query(int node, int l, int r, int x) {
        if (l == r) return tree[node](x);
        int mid = (l + r) >> 1; long long res = tree[node](x);
        if (x <= mid) res = max(res, query(node << 1, l, mid, x));
        else res = max(res, query(node << 1 | 1, mid + 1, r, x));
        return res;
    }
};
```

8.5 LineContainer (CHT)

Dynamic Convex Hull Trick for maximum query in $O(\log N)$. For Minimum: insert `add(-k, -m)` and negate output `-query(x)`.

```
/* Hash: 21066535 */
struct Line {
    mutable long long k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(long long x) const { return p < x; }
};

struct LineContainer : set<Line, less<>> {
    static const long long INF = LLONG_MAX;

    long long div(long long a, long long b) { return a / b - ((a ^
     b) < 0 && a % b); }

    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = INF; return false; }
        if (x->k == y->k) x->p = x->m > y->m ? INF : -INF;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }

    void add(long long k, long long m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p) isect(x,
     erase(y));
    }

    long long query(long long x) {
        assert(!empty()); auto l = *lower_bound(x); return l.k * x
     + l.m;
    }
};
```

8.6 Minkowski Sum

Computes the Minkowski sum of two convex polygons in $O(N + M)$ time.

```
/* Hash: FA3EBFD4 */
void reorder(std::vector<Point>& p) {
    size_t pos = 0;
    for (size_t i = 1; i < p.size(); ++i) {
        if (p[i].y < p[pos].y || (p[i].y == p[pos].y && p[i].x < p[
     pos].x)) pos = i;
    }
    std::rotate(p.begin(), p.begin() + pos, p.end());
}
std::vector<Point> minkowskiSum(std::vector<Point> A, std::vector<
     Point> B) {
    reorder(A); reorder(B);
    A.push_back(A[0]); A.push_back(A[1]);
    B.push_back(B[0]); B.push_back(B[1]);
    std::vector<Point> res;
    size_t i = 0, j = 0;
    while (i < A.size() - 2 || j < B.size() - 2) {
        res.push_back(A[i] + B[j]);
        long long cross = (A[i + 1] - A[i]).cross(B[j + 1] - B[j]);
        if (cross >= 0 && i < A.size() - 2) ++i;
        if (cross <= 0 && j < B.size() - 2) ++j;
    }
    return res;
}
```

8.7 Polygon Area (Shoelace Formula)

Computes the area of a polygon in $O(N)$ given its vertices ordered sequentially. Uses the global `Point` and `cross` from Geometry Template.

```
/* Hash: C632B83D */
double polygon_area(const vector<Point>& pts) {
    long long area2 = 0; int n = pts.size();
    for (int i = 0; i < n; i++) {
        area2 += cross(pts[i], pts[(i + 1) % n]);
    }
    return abs(area2) / 2.0;
}
```

8.8 Smallest Enclosing Circle (Welzl's Algorithm)

Finds the minimum bounding circle in $O(N)$ expected time. Uses global `Point`, `dist`, and `rng`.

```
/* Hash: 18086E7E */
struct Circle { Point center; double r; };

bool is_inside(Point p, Circle c) { return dist(p, c.center) <= c.r
     + 1e-9; }

Circle circle_from_3_points(Point A, Point B, Point C) {
    double bx = B.x - A.x, by = B.y - A.y, cx = C.x - A.x, cy = C.y
     - A.y;
    double BB = bx * bx + by * by, CC = cx * cx + cy * cy, D = 2 *
     (bx * cy - by * cx);
    Point center = {A.x + (cy * BB - by * CC) / D, A.y + (bx * CC -
     cx * BB) / D};
    return {center, dist(center, A)};
```

```
}

Circle circle_from_2_points(Point A, Point B) {
    return {{(A.x + B.x) / 2.0, (A.y + B.y) / 2.0}, dist(A, B) /
     2.0};
}

Circle welzl(vector<Point>& pts, vector<Point> r, int n) {
    if (n == 0 || r.size() == 3) {
        if (r.empty()) return {{0, 0}, 0};
        if (r.size() == 1) return {r[0], 0};
        if (r.size() == 2) return circle_from_2_points(r[0], r[1]);
        return circle_from_3_points(r[0], r[1], r[2]);
    }
    Point p = pts[n - 1]; Circle c = welzl(pts, r, n - 1);
    if (is_inside(p, c)) return c;
    r.push_back(p); return welzl(pts, r, n - 1);
}

Circle get_smallest_enclosing_circle(vector<Point> pts) {
    shuffle(pts.begin(), pts.end(), rng);
    return welzl(pts, {}, pts.size());
}
```

9 Misc

9.1 Custom Hash for hashmap & hashset

```
/* Hash: 541F4138 */
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::
     now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

To use this structure, pass `custom_hash` as the third template argument when defining your hash container:

```
/* Hash: 1F4ACD88 */
unordered_map<long long, int, custom_hash> safe_map;
unordered_set<long long, custom_hash> safe_set;
```

9.2 Hill Climbing (Heuristics)

A local search meta-heuristic that continuously moves in the direction of increasing/decreasing value (ascending the hill) to find an optimal state. Unlike Simulated Annealing, it is purely greedy and can get trapped in local optima.
Core Methodology:

1. **Generate Initial State:** Construct a valid random configuration and calculate its objective cost function score.
2. **Evaluate Neighbors:** Generate small mutations or modifications to the active parameters (e.g., swapping two elements).
3. **Greedy Step:** Compare the mutated score with the original. If the mutant state improves the objective criteria, instantly accept it as the new baseline. Otherwise, discard it immediately.
4. **Termination:** Repeat until no neighboring modifications yield any further improvements (local peak reached), or the allocated execution time limit expires.

Strategies to Escape Local Optima (Random Restart): Because pure Hill Climbing stops at the very first local peak it encounters, combine it with a **Random Restart** strategy inside competitive programming constraints:

- Enclose the entire hill climbing block within an external loop or time-guard window (`while (clock() < 0.95 * CLOCKS_PER_SEC)`).
- Whenever a state becomes completely stagnant (no mutations improve the score anymore), save its peak metrics to a global maximum tracker.
- Fully randomize the current state configuration back to zero and launch a brand new hill climbing ascent from that arbitrary coordinate.

9.3 Simulated Annealing (Heuristics)

Randomized optimization for NP-hard problems. Escapes local minima by accepting worse states with a temperature-dependent probability $P = e^{-\Delta/T}$.

```
/* Hash: DE9A7394 */
inline double get_rand() { return (double)rng() / mt19937::max(); }

struct State {
    long long cost;
    void init() {}   // Generate initial state & compute cost
    void mutate() {} // Apply small random modification & update
     cost
    void revert() {} // Undo mutation if rejected
};

void simulated_annealing() {
    State curr; curr.init();
    State best = curr;

    double T = 1e5, T_min = 1e-3, alpha = 0.9995; // Adjust
     parameters based on problem
    auto start_time = chrono::steady_clock::now();

    while (T > T_min) {
        // TLE Protection: Exit just before 1.0s time limit
        auto elapsed = chrono::duration<double>(chrono::
     steady_clock::now() - start_time).count();
        if (elapsed > 0.95) break;

        long long old_cost = curr.cost;
        curr.mutate();
        long long diff = curr.cost - old_cost; // Negative means
     improvement (Minimization)

        // Metropolis Criterion: Accept if better, or with
     probability exp(-diff / T)
        if (diff < 0 || exp(-diff / T) > get_rand()) {
            if (curr.cost < best.cost) best = curr;
        } else {
            curr.revert();
        }
        T *= alpha; // Cool down
    }
}
```

9.4 ModInt Template

```
/* Hash: 197B492C */
template<int MOD>
struct ModInt {
    int v;
    ModInt() : v(0) {}
    ModInt(long long _v) { v = int((-MOD < _v && _v < MOD) ? _v :
     _v % MOD); if (v < 0) v += MOD; }

    explicit operator int() const { return v; }
    bool operator==(const ModInt& o) const { return v == o.v; }
    bool operator!=(const ModInt& o) const { return v != o.v; }

    ModInt& operator+=(const ModInt& o) { v += o.v; if (v >= MOD) v
      -= MOD; return *this; }
    ModInt& operator-=(const ModInt& o) { v -= o.v; if (v < 0) v +=
      MOD; return *this; }
    ModInt& operator*=(const ModInt& o) { v = int((long long)v * o.
     v % MOD); return *this; }

    ModInt pow(long long p) const {
        ModInt res = 1, base = *this;
        for (; p > 0; p >>= 1, base *= base) if (p & 1) res *= base
     ;
        return res;
    }
    ModInt inv() const { return pow(MOD - 2); } // Requires MOD to
     be a prime
    ModInt& operator/=(const ModInt& o) { return *this *= o.inv();
     }

    friend ModInt operator+(ModInt a, const ModInt& b) { return a
     += b; }
    friend ModInt operator-(ModInt a, const ModInt& b) { return a
     -= b; }
    friend ModInt operator*(ModInt a, const ModInt& b) { return a
     *= b; }
    friend ModInt operator/(ModInt a, const ModInt& b) { return a
     /= b; }
};
using mint = ModInt<1000000007>;
```

- Use `mint` to declare variables. For example: `mint a = 5, b = 10;`
- Supports all standard arithmetic operators (`+`, `-`, `*`, `/`), compound assignments (`+=`, `-=`, `*=`, `/=`), and comparisons (`==`, `!=`).
- Call `a.pow(p)` to compute $a^p \pmod{MOD}$ efficiently in $O(\log p)$.

- Call `a.inv()` to find the modular multiplicative inverse using Fermat's Little Theorem.

9.5 Policy-Based Data Structures (PBDS)

An extension structure in GCC that provides a high-performance Ordered Set, supporting index-based lookups (`find_by_order`) and order queries (`order_of_key`) in $O(\log N)$.

```
/* Hash: 2DAA724E */
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// Definition for Ordered Set
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
     tree_order_statistics_node_update>;

// Definition for Ordered Multiset (allows duplicate keys)
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>,
     rb_tree_tag, tree_order_statistics_node_update>;
```

Basic operations and functions usage pattern:

```
/* Hash: 7FAEB7C7 */
ordered_set<int> st;

// 1. Insert elements normally
st.insert(1); st.insert(4); st.insert(8);

// 2. find_by_order(k): Returns an iterator to the k-th smallest
      element (0-indexed)
auto it = st.find_by_order(1); // Points to '4'
int val = *it;

// 3. order_of_key(x): Returns the number of elements strictly
      smaller than x
int cnt = st.order_of_key(5); // Returns 2 (elements 1 and 4 are
      smaller)

// 4. Erase element in ordered_multiset (MUST use iterator to avoid
       removing all duplicates)
ordered_multiset<int> mst;
mst.insert(4); mst.insert(4);
mst.erase(mst.find_by_order(mst.order_of_key(4))); // Safely erases
       only one '4'
```

9.6 Stress Tester

Automated stress testing harness using identical execution checking paths. Run `main.cpp` to compile and test.

```
/* Hash: 4727B10A */
// checker.cpp
int32_t main() {
    ifstream f_user("user.out"), f_brute("brute.out");
    string s_user, s_brute;
    while (f_user >> s_user) {
        if (!(f_brute >> s_brute) || s_user != s_brute) return 1;
    }
    if (f_brute >> s_brute) return 1;
    return 0;
}
```

```
/* Hash: 2FD4429C */
// main.cpp
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count
     ());

int rand_range(int l, int r) {
    uniform_int_distribution<long long> dist(l, r);
    return dist(rng);
}

void gen_test() {
    ofstream out("test.in");
    out << rand_range(1, 100) << " " << rand_range(1, 100) << "\n";
}

int32_t main() {
    system("g++ -O3 user.cpp -o user");
    system("g++ -O3 brute.cpp -o brute");
    system("g++ -O3 checker.cpp -o checker");

    for (int t = 1; t <= 10000; ++t) {
        gen_test();
        system("./user < test.in > user.out");
        system("./brute < test.in > brute.out");
        if (system("./checker")) {
            cout << "WA on test " << t << "\n";
            return 0;
        }
    }
    cout << "All tests passed!\n";
    return 0;
}
```

10 Classic

10.1 Bounded Knapsack with Binary Grouping

Solves the Bounded Knapsack problem where each item i has a limited count C_i. Reduces the state space to a 0-1 Knapsack instance in $O(W \sum \log C_i)$ time complexity.

```
/* Hash: D4116C80 */
struct Item {
    long long w, v;
};

long long bounded_knapsack(int n, long long W, const vector<long
     long>& weight, const vector<long long>& value, const vector<
     int>& cnt) {
    vector<Item> items;
    for (int i = 0; i < n; ++i) {
        int rem = cnt[i];
        for (int k = 1; rem > 0; k <<= 1) {
            int take = min(k, rem);
            rem -= take;
            items.push_back({take * weight[i], take * value[i]});
        }
    }

    vector<long long> dp(W + 1, 0);
    for (const auto& item : items) {
        for (int c = W; c >= item.w; --c) {
            dp[c] = max(dp[c], dp[c - item.w] + item.v);
        }
    }
    return dp[W];
}
```

10.2 CSES Counting Tilings

Solves the grid tiling problem (CSES 2181) for an $N \times M$ grid using 1×2 and 2×1 dominoes in $O(N \cdot M \cdot 2^N)$ time and $O(2^N)$ space complexity.

```
/* Hash: D987FA77 */
const int MOD = 1e9 + 7;

int solve_tiling(int n, int m) {
    if (n > m) swap(n, m); // Optimize for the smaller dimension
    vector<long long> dp(1 << n, 0);
    dp[0] = 1;

    for (int j = 0; j < m; ++j) {
        for (int i = 0; i < n; ++i) {
            vector<long long> next_dp(1 << n, 0);
            for (int mask = 0; mask < (1 << n); ++mask) {
                if (!dp[mask]) continue;

                if (mask & (1 << i)) {
                    // Current cell is already covered by a
     vertical domino from the top row
                    next_dp[mask ^ (1 << i)] = (next_dp[mask ^ (1
     << i)] + dp[mask]) % MOD;
                } else {
                    // Option 1: Place a horizontal domino (extends
      to the right column)
                    next_dp[mask ^ (1 << i)] = (next_dp[mask ^ (1
     << i)] + dp[mask]) % MOD;

                    // Option 2: Place a vertical domino down (if
     not on the last row and bottom cell is empty)
                    if (i + 1 < n && !(mask & (1 << (i + 1)))) {
                        next_dp[mask ^ (1 << (i + 1))] = (next_dp[
     mask ^ (1 << (i + 1))] + dp[mask]) % MOD;
                    }
                }
            }
            dp = move(next_dp);
        }
    }
    return dp[0];
}
```

10.3 Closest Pair of Points

Finds the closest pair of points in $O(N \log N)$ time using a sweep-line algorithm with `std::set`. Uses global `Point` and `dist` from Geometry Template.

```
/* Hash: 32B496ED */
bool compare_y(const Point& a, const Point& b) { return a.y < b.y;
     }

pair<Point, Point> closest_pair(vector<Point>& pts) {
    int n = pts.size(); sort(pts.begin(), pts.end());
    double min_d = 1e18; pair<Point, Point> best_pair;
    set<Point, decltype(&compare_y)> active_set(&compare_y);

    int left = 0;
    for (int i = 0; i < n; i++) {
        while (pts[i].x - pts[left].x >= min_d) active_set.erase(
     pts[left++]);
        Point lower_bound_pt = {0, (long long)(pts[i].y - min_d)};
        auto it = active_set.lower_bound(lower_bound_pt);

        while (it != active_set.end() && it->y - pts[i].y < min_d)
     {
            double d = dist(pts[i], *it);
```

```
            if (d < min_d) { min_d = d; best_pair = {pts[i], *it};
    }
            it++;
        }
        active_set.insert(pts[i]);
    }
    return best_pair;
}
```

10.4 De Bruijn Sequence

Generates the shortest cyclic sequence of length B^K that contains every possible sequence of length K over an alphabet of size B as a substring exactly once.

```
/* Hash: AC272367 */
struct DeBruijn {
    int b, k;
    vector<int> sequence, a;

    DeBruijn(int _b, int _k) : b(_b), k(_k), a(_k * _b, 0) {}

    void db_dfs(int t, int p) {
        if (t > k) {
            if (k % p == 0) {
                for (int i = 1; i <= p; ++i) sequence.push_back(a[i
    ]);
            }
        } else {
            a[t] = a[t - p];
            db_dfs(t + 1, p);
            for (int i = a[t - p] + 1; i < b; ++i) {
                a[t] = i;
                db_dfs(t + 1, t);
            }
        }
    }

    vector<int> generate() {
        sequence.clear();
        db_dfs(1, 1);
        return sequence;
    }
};
```

11 Formulas

11.1 Essential Theorems

Critical theorems and properties for structural reductions and mathematical transformations.

Graph Theory Invariants & Reductions:

- **Dilworth:** Max antichain (no two comparable) = min chains to cover poset. Reduces to Max Bipartite Matching.
- **Konig:** In any bipartite graph, $|Max\ Matching| = |Min\ Vertex\ Cover|$.
- **Independent Set:** S is vertex cover iff $V \setminus S$ is independent set $\implies |Min\ Vertex\ Cover| + |Max\ Independent\ Set| = |V|$.
- **Gallai:** In graphs without isolated nodes, $|Max\ Matching| + |Min\ Edge\ Cover| = |V|$.
- **Eulerian Path:** Undirected: cycle iff all degrees even; path iff exactly 0 or 2 nodes have odd degree. Directed: cycle iff $in[u] = out[u]$ for all u; path iff at most one node has $out - in = 1$ and at most one has $in - out = 1$.

Number Theory Identities:

- **Euler & Fermat:** $\gcd(a, m) = 1 \implies a^{\phi(m)} \equiv 1 \pmod m$. If p is prime, $a^{p-1} \equiv 1 \pmod p$.
- **Bezout:** $\exists x, y$ such that $ax + by = \gcd(a, b)$, solved via ExtGCD.
- **Wilson:** Integer $n > 1$ is prime iff $(n-1)! \equiv -1 \pmod n$.
- **Mobius Inversion:** If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} \mu(d) g(n/d)$.
- **GCD Properties:**
  - $\gcd(a^n - 1, a^m - 1) = a^{\gcd(n,m)} - 1$.
  - $\gcd(F_a, F_b) = F_{\gcd(a,b)}$

11.2 Essential Formulas

Binet's formula (Fibonacci):

$$F_n = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$$

Recurrence ↔ Closed-form:

- **Recurrence → Closed-form:** For $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, solve the characteristic equation: $x^k - c_1 x^{k-1} - \cdots - c_k = 0$.
  - Distinct roots r_i: $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n$
  - Repeated root r (multiplicity m): $a_n = (\alpha_0 + \alpha_1 n + \cdots + \alpha_{m-1} n^{m-1}) r^n$
  - → Plug in base cases $(a_0, a_1, \dots)$ to solve a system of linear equations for α_i.
- **Closed-form → Recurrence:** Reverse the process. If $a_n = \alpha_1 (r_1)^n + \alpha_2 (r_2)^n$, then r_1, r_2 are roots of $(x - r_1)(x - r_2) = x^2 - (r_1 + r_2)x + r_1 r_2 = 0 \to a_n = (r_1 + r_2) a_{n-1} - (r_1 r_2) a_{n-2}$.

Combinatorics & Modular Queries: $C_n^k = \frac{n!}{k!(n-k)!} \pmod M$. Precompute factorials in $O(N)$. Linear inverse:

```
inv[i] = -(M / i) * inv[M % i] % M + M;
```

Vandermonde's Identity: $\sum_{k=0}^{r} C_m^k C_n^{r-k} = C_{m+n}^r$ (Choosing r items from two combined groups).

Fermat's Polygonal Number (Gauss EUREKA): Every $N > 0$ can be expressed as the sum of at most k polygonal numbers of side k ($\max \text{count} = k$). General formula for n-th polygonal number of side k:

$$P_k(n) = \frac{(k-2)n^2 - (k-4)n}{2} = n + (k-2) \cdot \frac{n(n-1)}{2}$$

- **Recurrence Relation between Sides:** A k-polygonal number can be derived from a $(k-1)$-polygonal number by adding the $(n-1)$-th triangular number ($T_{n-1} = \frac{n(n-1)}{2}$):

$$P_k(n) = P_{k-1}(n) + T_{n-1} = P_{k-1}(n) + \frac{n(n-1)}{2}$$

- **Triangular Decomposition:** Any k-polygonal number can be broken down directly into a combination of the n-th and $(n-1)$-th triangular numbers:

$$P_k(n) = T_n + (k-3) \cdot T_{n-1}$$

- **General $O(1)$ Check / Inverse Function:** To check if N is a k-polygonal number, solve $(k-2)n^2 - (k-4)n - 2N = 0$. Let $\Delta = (k-4)^2 + 8N(k-2)$. N is valid iff Δ is a perfect square and $n = \frac{(k-4)+\sqrt{\Delta}}{2(k-2)}$ is an integer.
- **Find side k from N and n:** $k = 2 + \frac{2(N-n)}{n(n-1)}$.

Catalan Numbers: (BSTs, valid parenthesization, non-crossing polygon dissections).
$C_0 = 1$, $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i} = \frac{1}{n+1} C_{2n}^n = C_{2n}^n - C_{2n}^{n-1}$ Terms: $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862$.

Derangements: (Permutations with no fixed points).
$D_1 = 0$, $D_2 = 1$, $D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n$ Terms: $0, 1, 2, 9, 44, 265, 1854, 14833, 133496$.

Stirling Numbers of the 2nd Kind: (Partition n labeled items into k unlabeled non-empty subsets).
$S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$. Base:
$S(0, 0) = 1$, $S(n, 0) = S(0, n) = 0$. Partition n labeled items into k labeled boxes $= k! \cdot S(n, k)$.

Lucas' Theorem: $C_n^k \pmod p$ for small prime p $(n, k \le 10^{18})$. Base-p representations:

$n = \sum n_i p^i$, $k = \sum k_i p^i \implies C_n^k \equiv \prod C_{n_i}^{k_i} \pmod p$ (where $C_{n_i}^{k_i} = 0$ if $n_i < k_i$).

Burnside's Lemma: Number of orbits (distinct objects under symmetry) $= \frac{1}{|G|} \sum_{g \in G} |X^g|$ where $|G|$ is size of symmetry group, and $|X^g|$ is number of elements fixed by symmetry g.

Graph Theory Counting:

- **Cayley's Formula:** Labeled spanning trees in $K_n = n^{n-2}$. In a forest of k trees with roots $r_1..r_k$ covering all n nodes, ways to connect them into a single tree $= n^{k-2} \cdot \prod \text{sz}(T_i)$.
- **Euler's Planar Formula:** Connected planar graph: $V - E + F = 2$.
- **Matrix Tree Theorem:** Let A be adjacency matrix, D degree matrix. Kirchhoff matrix $L = D - A$. Delete row i and col i. Determinant of resulting matrix = number of spanning trees.

Geometry Theorems:

- **Pick's Theorem:** Grid polygon area $A = I + B/2 - 1$. (I: interior points, B: boundary points).
- **Boundary Points:** Integer points on segment (x_1, y_1) to (x_2, y_2) inclusive $= \gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$.

11.3 Cheatsheet

A comprehensive reference table for prime counts, highly composite numbers, primorials, factorials, and modular bounds.

1. **Number of Primes $\pi(n)$:**

n	$\pi(n)$
10^2	25
10^3	168
10^4	1 229
10^5	9 592
10^6	78 498
10^7	664 579
10^8	5 761 455
10^9	50 847 534
10^{10}	455 052 511

2. **Highly Composite Numbers:**

$\leq n$	x	$d(x)$
10^3	840	32
10^4	7 560	64
10^5	83 160	128
10^6	720 720	240
10^7	8 648 640	448
10^8	73 513 440	768
10^9	735 134 400	1 344
10^{10}	7 351 440 000	2 304
10^{11}	69 837 768 000	4 032
10^{12}	963 761 198 400	6 720
10^{13}	8 673 850 785 600	10 752
10^{14}	95 412 358 641 600	17 280
10^{15}	858 711 227 774 400	26 880
10^{16}	9 445 823 505 518 400	41 472
10^{17}	93 512 652 704 632 000	64 512
10^{18}	841 613 874 341 688 000	103 680

3. **Primorials ($n\#$):**

n	$n\#$
2	2
3	6
5	30
7	210
11	2 310
13	30 030
17	510 510
19	9 699 690
23	223 092 870
29	6 469 693 230
31	200 560 490 130
37	7 420 738 134 810
41	304 250 263 527 210
43	13 082 761 331 670 030
47	614 889 782 588 491 410

4. **Factorials ($n!$):**

n	$n!$
1	1
2	2
3	6
4	24
5	120
6	720
7	5 040
8	40 320
9	362 880
10	3 628 800
11	39 916 800
12	479 001 600
13	6 227 020 800
14	87 178 291 200
15	1 307 674 368 000
16	20 922 789 888 000
17	355 687 428 096 000
18	6 402 373 705 728 000
19	121 645 100 408 832 000
20	2 432 902 008 176 640 000

5. **Hashing Bases & NTT Modulos:**

Type	Value (Prime)	g	NTT
Hash Base	31, 37, 131, 13331	—	—
Hash Base	163, 233, 10007	—	—
Std Mod	$10^9 + 7$	5	Non
Std Mod	$10^9 + 9$	13	Non
Big Mod	$10^{18} + 9$	—	Non
NTT Mod	998244353	3	2^{23}
NTT Mod	167772161	3	2^{25}
NTT Mod	469762049	3	2^{26}

6. **Data Type Limits:**

Type	Max Value	Safe Geo
32-bit `int`	$2,147,483,647$ ($\approx 2 \times 10^9$)	$X, Y \leq 30\,000$
64-bit `ll`	9.22×10^{18}	$X, Y \leq 2 \times 10^9$
`__int128`	$\approx 1.7 \times 10^{38}$	Max cross prod

7. **Sequence Limits:**

Seq.	Max 32-bit	Max 64-bit
Fibonacci	$F_{46} \approx 1.83 \times 10^9$	$F_{92} \approx 7.54 \times 10^{18}$
Catalan	$C_{19} \approx 1.76 \times 10^9$	$C_{33} \approx 3.32 \times 10^{18}$

8. **Powers Quick Reference:**

2^n	Exact Value	10^n	Pref.
2^{10}	1 024	10^3	K
2^{20}	1 048 576	10^6	M
2^{30}	1 073 741 824	10^9	G
2^{40}	1.09×10^{12}	10^{12}	T
2^{50}	1.12×10^{15}	10^{15}	P
2^{60}	1.15×10^{18}	10^{18}	E

9. **Terminal Shortcuts**

Maximize stack size `ulimit -s unlimited`
Perm. stack size (Bash) `echo "ulimit -s unlimited" » ~/.bashrc`
Kill hanging program `Ctrl+C`
Redirect I/O for testing `./a.out < in > out`
Compare outputs (ignore space) `diff -w out sol`

11.4 Vocabulary

Arbitrary
Bất kỳ, không ràng buộc.
Matching
Cặp ghép (tập cạnh không chung đỉnh).
Maximal Matching
Cặp ghép tối đại (không thể thêm cạnh).
Maximum Matching
Cặp ghép cực đại (nhiều cạnh nhất).
Perfect Matching
Cặp ghép hoàn hảo (phủ kín mọi đỉnh).
Independent Set
Tập độc lập (tập đỉnh không có cạnh nối).
Vertex Cover
Phủ đỉnh (tập đỉnh chạm vào mọi cạnh).
Edge Cover
Phủ cạnh (tập cạnh chạm vào mọi đỉnh).
Idempotent
Lũy đẳng ($f(x, x) = x \to$ GCD, Max, Min).
Mutually exclusive
Xung khắc (không thể xảy ra cùng lúc).
Complement
Phần bù / Đồ thị bù (đảo ngược cạnh).
Bijective
Song ánh (ánh xạ 1-1 tương ứng hoàn toàn).
Invariants
Đại lượng bất biến (giá trị không đổi qua các thao tác).
Collinear
Thẳng hàng (các điểm cùng nằm trên 1 đường thẳng).
Coplanar
Đồng phẳng (các điểm cùng thuộc 1 mặt phẳng).
Cross product
Tích có hướng (tìm hướng quay CCW/CW, tính diện tích).
Dot product
Tích vô hướng (tính góc, độ dài hình chiếu).
Perpendicular / Orthogonal
Vuông góc / Trực giao (dot product = 0).
Counter-clockwise (CCW)
Ngược chiều kim đồng hồ (cross product > 0).
Clockwise (CW)
Thuận chiều kim đồng hồ (cross product < 0).
Incircle
Đường tròn nội tiếp (tiếp xúc trong với các cạnh của đa giác).
Incenter
Tâm đường tròn nội tiếp (giao điểm các đường phân giác trong).
Inradius (r)
Bán kính đường tròn nội tiếp (Tính nhanh: $r = \text{Area}/s$ với s là nửa chu vi).
Circumcircle
Đường tròn ngoại tiếp (đi qua tất cả các đỉnh của đa giác).
Circumcenter
Tâm đường tròn ngoại tiếp (giao điểm các đường trung trực).
Circumradius (R)
Bán kính đường tròn ngoại tiếp (Tam giác: $R = abc/(4 \cdot \text{Area})$).
Excircle / Excenter
Đường tròn bàng tiếp / Tâm bàng tiếp (tiếp xúc 1 cạnh và 2 cạnh kéo dài).
Concyclic
Đồng viên (các điểm cùng nằm trên một đường tròn).
Tangent
Tiếp tuyến (đường thẳng tiếp xúc đường tròn tại 1 điểm, vuông góc bán kính).
Secant
Cát tuyến (đường thẳng cắt đường tròn tại 2 điểm phân biệt).
Chord
Dây cung (đoạn thẳng nối 2 điểm trên đường tròn).
Centroid
Trọng tâm (giao điểm các đường trung tuyến; hoặc trung bình cộng tọa độ đa giác).
Orthocenter
Trực tâm (giao điểm các đường cao của tam giác).
Sector / Segment
Hình quạt tròn (giới hạn bởi 2 bán kính và cung) / Hình viên phân (giới hạn bởi dây cung và cung).
Eulerian / Hamiltonian
Đồ thị Euler (có chu trình qua mọi cạnh) / Đồ thị Hamilton (có chu trình qua mọi đỉnh).

TRÌNH - HIEUTHUHAI

Tiếng Việt

Ey ey yeah yeah
Ối zồi ôi ối zồi ôi
Trình là gì mà là trình ai chấm
Anh chỉ biết làm ba mẹ tự hào
Xây căn nhà thật to ở một mình hai tấm
Ối zồi ôi ối zồi ôi
Cứ lên mạng phán xét tưởng là mình oai lắm
Nhìn vào sự nghiệp anh thèm chảy nước miếng
Giống mấy thằng biến thái nó đang rình ai tắm
Mua bao nhiêu căn nhà (đếm được đếm được)
Năm bao nhiêu show (đếm được đếm được)
Bao nhiêu bài hit (đếm được đếm được)
Bao nhiêu trong bank (đếm được đếm được)
Mua bao nhiêu căn nhà (đếm được đếm được)
Năm bao nhiêu show (đếm được đếm được)
Bao nhiêu là cúp (đếm được đếm được)
Có bao nhiêu fan (không đếm được không đếm được)

English

Ey ey yeah yeah
Oh my god oh my god
What is skill, and who has the right to judge?
I only know how to make my parents proud
Building a huge 2-story house to live by myself
Oh my god oh my god
Keep judging online, thinking you're so cool
Staring at my career, drooling with envy
Just like perverts peeping at someone taking a bath
How many houses bought? (countable, countable)
How many shows a year? (countable, countable)
How many hit songs? (countable, countable)
How much in the bank? (countable, countable)
How many houses bought? (countable, countable)
How many shows a year? (countable, countable)
How many trophies? (countable, countable)
How many fans do I have? (uncountable, uncountable!)

HSG-sicula